



Maskininlärningsbaserad Objektlokalisering i Planritningar

Ett första steg mot automatisering av bedömningsprocessen

Machine Learning-Based Object Localization in Floor Plans

Kandidatarbete inom civilingenjörsutbildningen vid Chalmers

Jacob Nejderås

Victor Valjus

Villiam Kroon

Maskininlärningsbaserad Objektlokalisering i Planritningar

Ett första steg mot automatisering av bedömningsprocessen

Kandidatarbete i matematik inom civilingenjörsprogrammet Teknisk Matematik

Victor Valjus

Kandidatarbete i matematik inom civilingenjörsprogrammet Automation och mekatronik vid Chalmers

Jacob Nejderås Villiam Kroon

Handledare: Maria Roginskaya
Larisa Beilina

Institutionen för Matematiska vetenskaper
CHALMERS TEKNISKA HÖGSKOLA
GÖTEBORGS UNIVERSITET
Göteborg, Sverige 2023

Förord

Detta arbete har genomförts under våren 2026 vid Institutionen för Matematiska vetenskaper på Chalmers Tekniska Högskola och Göteborgs Universitet. Gruppen vill först och främst tacka vår handledare Maria Roginskaya för vägledning under projektet. Vi vill även rikta ett stort tack till vår bi-handledare Larisa Beilina för hjälp med detaljer och matematiskt notation vid rapportskrivning. Vi vill även skicka med ett tips till de som läser denna rapport. Det har varit till stor hjälp att läsa tidigare kandidatarbeten för att hämta inspiration till metoder, utformning och tips under hela arbetets gång.

Arbetet har dokumenterats i form av en gemensam dagbok och tidslogg som har skrivits varje vecka i ett roterande schema mellan gruppmedlemmarna. Eftersom gruppens storlek var mindre har inte arbetsuppgifterna delats upp utan uppdelningen har varit relativt flytande genom hela arbetet. Hela arbetet, bortsett från rapportskrivning, har gjorts genom kodning av olika moduler. Victor och Villiam har arbetat med kodning av bearbetnings- och jämförelsealgoritmer, medan Jacob har ansvarat för körning och resultatinhämtning. Villiam har haft huvudansvaret för kontakt med externa parter och projektledning. I Tabell 1 finns en förteckning över vem som är skaparen av figurerna som återfinns i rapporten. I Tabell 2 presenteras en fullständig genomgång av rapportens innehåll och ansvarig person för respektive del. Innan inlämning har hela gruppens tillsammans läst igenom hela rapporten för att göra justeringar och bearbetning för att få en sammanhängande text.

Tabell 1: Förteckning över skapare av figurer

<i>Avsnitt</i>	<i>Figur</i>	<i>Skapad av</i>
<i>1.2</i>	<i>Figur 1</i>	<i>Villiam</i>
<i>2.1</i>	<i>Figur 2</i>	<i>Villiam</i>
<i>2.3</i>	<i>Figur 3</i>	<i>Villiam</i>
<i>2.5</i>	<i>Figur 4</i>	<i>Villiam</i>
<i>2.5</i>	<i>Figur 5</i>	<i>Villiam</i>
<i>3.1.2</i>	<i>Figur 6</i>	<i>Villiam</i>
<i>3.1.3</i>	<i>Figur 7</i>	<i>Villiam</i>
<i>3.1.4</i>	<i>Figur 8</i>	<i>Victor</i>
<i>3.3</i>	<i>Figur 9</i>	<i>Victor</i>
<i>4.2.1</i>	<i>Figur 10</i>	<i>Villiam</i>
<i>4.2.2</i>	<i>Figur 11</i>	<i>Villiam</i>
<i>D.1</i>	<i>Figur 12</i>	<i>Victor</i>
<i>D.1</i>	<i>Figur 13</i>	<i>Victor</i>
<i>D.2</i>	<i>Figur 14</i>	<i>Jacob</i>
<i>D.2</i>	<i>Figur 15</i>	<i>Jacob</i>

Tabell 2: Bidragsrapport

Kapitel	Rubrik	Författare
	<i>Förord</i>	–
	<i>Sammandrag</i>	–
	<i>Abstract</i>	–
<i>1</i>	<i>Inledning</i>	<i>Villiam</i>
<i>1.1</i>	<i>Maskininlärning</i>	<i>Villiam</i>
<i>1.1.1</i>	<i>Neurala Nätverk</i>	<i>Villiam</i>
<i>1.1.2</i>	<i>Klassiska modeller</i>	<i>Villiam</i>
<i>1.2</i>	<i>Syfte</i>	<i>Villiam</i>
<i>2</i>	<i>Teori</i>	<i>Villiam</i>
<i>2.1</i>	<i>Digital representation av en bild</i>	<i>Villiam</i>
<i>2.2</i>	<i>Inter Area Interpolation</i>	<i>Villiam</i>
<i>2.3</i>	<i>Hausdorff-avstånd</i>	<i>Villiam</i>
<i>2.4</i>	<i>Modifierat Hausdorff-avstånd</i>	<i>Villiam</i>
<i>2.5</i>	<i>Singularvärdesuppdelning</i>	<i>Villiam</i>
<i>2.6</i>	<i>Rotation av bilder</i>	<i>Villiam</i>
<i>2.7</i>	<i>Template matchning</i>	<i>Villiam</i>
<i>3</i>	<i>Metod</i>	<i>Villiam</i>
<i>3.1</i>	<i>Implementation</i>	<i>Villiam</i>
<i>3.1.1</i>	<i>Svepande fönster</i>	<i>Villiam</i>
<i>3.1.2</i>	<i>Non Maximum Supression</i>	<i>Villiam</i>
<i>3.1.3</i>	<i>Interpolation</i>	<i>Villiam</i>
<i>3.1.4</i>	<i>Negativ Padding</i>	<i>Victor</i>
<i>3.1.5</i>	<i>Normering</i>	<i>Victor</i>
<i>3.1.6</i>	<i>Klassificering</i>	<i>Villiam</i>
<i>3.2</i>	<i>Data</i>	<i>Victor, Jacob</i>
<i>3.3</i>	<i>Test av metoder</i>	<i>Victor</i>
<i>4</i>	<i>Resultat</i>	<i>Jacob</i>
<i>4.1</i>	<i>Numeriska resultat</i>	<i>Jacob</i>
<i>4.2</i>	<i>Tolkning av resultat</i>	<i>Jacob, Villiam</i>
<i>4.2.1</i>	<i>Normerat modifierat Hausdorff-avstånd</i>	<i>Jacob</i>
<i>4.2.2</i>	<i>Normerat template matchning</i>	<i>Jacob, Villiam</i>
<i>4.3</i>	<i>Kantdetektion som förbearbetningsmetod</i>	<i>Victor</i>
<i>4.4</i>	<i>Resultat CNN</i>	<i>Jacob</i>
<i>4.4.1</i>	<i>Bakgrund</i>	<i>Jacob</i>
<i>4.4.2</i>	<i>Resultat</i>	<i>Jacob</i>
<i>5</i>	<i>Diskussion</i>	<i>Jacob</i>
<i>5.1</i>	<i>Jämförelse av metoderna</i>	<i>Jacob, Victor</i>
<i>5.1.1</i>	<i>Begränsningar i data</i>	<i>Jacob</i>
<i>5.2</i>	<i>Framtida metoder</i>	<i>Victor</i>
<i>5.2.1</i>	<i>SVD som förbearbetningsmetod</i>	<i>Jacob</i>
<i>5.2.2</i>	<i>Tidsoptimerad kod</i>	<i>Jacob</i>
<i>5.3</i>	<i>Slutsats</i>	<i>Jacob</i>
<i>6</i>	<i>Samhälleliga och etiska aspekter</i>	<i>Villiam</i>
<i>A</i>	<i>Appendix 1 - Teori</i>	–
	<i>Algoritm 1</i>	<i>Villiam</i>
	<i>Sats 1</i>	<i>Villiam</i>
	<i>Sats 2</i>	<i>Villiam</i>
	<i>Sats 3</i>	<i>Villiam</i>
<i>B</i>	<i>Appendix 2 - källkod</i>	
<i>B.1</i>	<i>Hausdorff</i>	<i>Victor, Villiam</i>
<i>B.2</i>	<i>Kantdetektion och gaussisk utjämning</i>	<i>Victor, Villiam</i>
<i>B.3</i>	<i>Sliding window</i>	<i>Victor, Villiam, Jacob</i>
<i>B.4</i>	<i>Template matchning</i>	<i>Victor, Villiam</i>

Sammandrag

Forskningsgruppen för Byggnadsdesign vid *Chalmers Tekniska högskola* och liknande forskningsgrupper vid andra universitet i Sverige genomför analyser av bygglovshandlingar utifrån olika perspektiv. Idag är arbetet enbart en manuell process. Med dagens teknik är detta ett arbete som till stor del kan automatiseras genom objektidentifiering. Eftersom det saknas en databas med markerade bilder inom detta område är objektigenkänning krävs en analys av vilka metoder som kan användas för att markera objekt i dessa ritningar.

Eftersom syftet är att skapa ett ramverk för objektlokalisering avgränsades arbetet till att markera spisar i planritningar. Detta för att kunna analysera de valda metoderna utefter liknande förutsättningar. Arbetet undersöker klassiska bildbehandlingsmetoder såsom Hausdorff-avstånd och Template Matchning samt olika varianter och implementationer av dessa. Utöver de jämförande metoderna prövas ett antal förbearbetningsmetoder såsom svepande fönster, negativ padding, kantdetektering, interpolation och Non-maximum Supression.

Som dataset används ett urval av planritningar från bygglovshandlingar från Göteborgs stad under perioden 2022-2025 där olika varianter av spisar återfanns. För att tydligare kunna utvärdera de valda metoderna beskärdes bilderna så att det fanns exakt en spis i varje bild. Således avgränsades klassificeringsmetoden så att det antogs att det högst korrelerade området i bilden var spisen. Genom att använda samma dataset och prestandamått för alla algoritmer kunde de utvärderas på ett jämförbart sätt.

Template Matchnings metoden `TM_COEFF_NORMED` med negativ padding visar högst prestanda i undersökningen och den visar även på tydligast skillnad mellan sant-positiva och falskt-positiva klassificeringar. Utöver de ovan nämnda metoderna testades även att träna ett Konvolutionellt Nerualt Nätverk på ett egenmarkerat dataset. Detta visade på mycket bra prestanda och kan användas som en riktlinje på hur stort antal markerade bilder som kan behövas i en framtida automatiserad process.

Abstract

The Building Design research group at *Chalmers University of Technology*, together with similar research groups at other universities in Sweden, conducts analyses of building permit documents from various perspectives. At present, this work is carried out entirely manually. With current technology, however, much of this process can be automated through object detection. Since there is no existing database of annotated images in this domain, a study of suitable methods for annotating objects in these drawings is required to enable object recognition.

As the purpose of this work was to develop a framework for object localization, the scope was limited to identifying stoves in floor plans. This made it possible to evaluate the selected methods under comparable conditions. The study investigates classical image processing methods such as the Hausdorff distance and template matching, as well as various variations and implementations of these techniques. In addition to the comparative methods, several preprocessing techniques were tested, including sliding window, negative padding, edge detection, interpolation, and non-maximum suppression.

The dataset consisted of a selection of floor plans from building permit documents provided by the City of Gothenburg for the period 2022–2025, in which different variants of stoves were identified. To facilitate a clearer evaluation of the selected methods, the images were cropped so that each image contained exactly one stove. As a result, the classification task was defined under the assumption that the region with the highest correlation in the image corresponded to the stove. By using the same dataset and performance metrics for all algorithms, they could be evaluated in a consistent and comparable manner.

The template matching method `TM_COEFF_NORMED` with negative padding achieved the highest performance in the study and also showed the clearest distinction between true-positive and false-positive classifications. In addition to the methods described above, a Convolutional Neural Network (CNN) was trained on a custom-annotated dataset. This approach demonstrated very strong performance and can serve as an indication of the number of annotated images that may be required for a future automated process.

Innehåll

1	Inledning	1
1.1	Maskininlärning	1
1.1.1	Neurala nätverk	1
1.1.2	Klassiska modeller	2
1.2	Syfte	2
2	Teori	3
2.1	Digital representation av en bild	3
2.2	Inter Area Interpolation	4
2.3	Hausdorff-avstånd	4
2.4	Modifierat Hausdorff-avstånd	5
2.5	Singularvärdesuppdelning (SVD)	5
2.6	Rotation av bilder i två dimensioner	7
2.7	Template matchning	8
3	Metod	8
3.1	Implementation	8
3.1.1	Svepande fönster	9
3.1.2	Non Maximum Supression	9
3.1.3	Interpolation	10
3.1.4	Negativ Padding	11
3.1.5	Normering	11
3.1.6	Klassificering	11
3.2	Data	12
3.3	Test av metoder	12
4	Resultat	13
4.1	Numeriska resultat	13
4.2	Tolkning av resultat	14
4.2.1	Normerat modifierat Hausdorff-avstånd	14
4.2.2	Normerat template matchning	15
4.3	Kantdetektion som förbearbetningsmetod	16
4.4	Resultat CNN	16
4.4.1	Bakgrund	16
4.4.2	Resultat	16
5	Diskussion	16
5.1	Jämförelse av metoderna	16
5.1.1	Begränsningar i data	17
5.2	Framtida metoder	17
5.2.1	SVD som förbearbetningsmetod	18
5.2.2	Tidsoptimerad kod	18
5.3	Slutsats	18
6	Samhälleliga och etiska aspekter	19
A	Appendix 1 – teori	i
A.1	Modifierat Hausdorff-avstånd	i
A.2	Template matching algoritm	i
B	Appendix 2 – källkod	iii
B.1	Hausdorff	iii
B.2	Kantdetektion och gaussisk utjämning	iii
B.3	Sliding window	iv
B.4	Template matchning	x

C	Appendix 3 – exempelritning	xii
D	Appendix 4 – markerade resultat bilder	xiv
D.1	Kantdetektion	xiv
D.2	CNN	xvi

1 Inledning

Industrier, företag och samhället i stort är i konstant förändring. Med start under 1960-talet har samhället sedan dess genomgått en stor digitaliseringsfas. Papper har bytts ut till filer, post har bytts ut till elektroniska meddelanden och dokumentering har flyttats till molnet. Digitaliseringen har lett till att stora mängder bilder genereras och lagras elektroniskt. Bilder kan matematiskt representeras diskreta punktmängder, vilket möjliggör analys med hjälp av metriska rum och avstånd.

Den industriella revolutionen medförde en betydande ökning i levnadsstandard, produktivitet och produktionskapacitet genom införandet av maskiner. Den nuvarande utvecklingsfasen benämns ofta som den fjärde industriella revolutionen, efter den första [2]. Termen Industry 4.0 introducerades i Tyskland omkring 2011 som en del av en strategisk satsning på digitalisering inom industrin. Begreppet syftar på integrationen av digital teknik, där datorer och system kommunicerar med varandra, vilket möjliggör automatisering av arbetsuppgifter som tidigare utfördes av människor och bidrar till ökad ekonomisk produktion.

Forskningsgruppen för Byggnadsdesign vid Chalmers Tekniska Högskola och liknande arbetsgrupper vid andra universitet i Sverige genomför ett arbete där stora resurser läggs på att undersöka planritningar för bygglov utefter olika perspektiv såsom levnadsstandard, tillgänglighet och rumslik effektivitet. I dagsläget är detta en process som i sin helhet genomförs utav människor med hög kompetens. Med andra ord, är det en kostsam process - både ekonomiskt och tidsmässigt. Detta motiverar en undersökning hur delar av, eller hela, processen dra nytta av automatisering som ett beslutsunderlag eller stöd vid bedömning.

1.1 Maskininlärning

Maskininlärning är ett delområde inom artificiell intelligens som fokuserar på att utveckla metoder som kan lära sig från data och göra prediktioner utan att explicit programmeras för en specificerad uppgift. Genom att identifiera mönster och samband i data kan dessa modeller användas för att lösa komplexa problem inom en rad olika områden, exempelvis bildanalys.

En del av bildanalys är objektigenkänning och objektlokalisering. Objektigenkänning avser uppgiften att klassificera en enskild bild, medan objektlokalisering syftar till att identifiera var i en bild ett visst objekt befinner sig, genom att ange dess position i en ruta. Dessa två uppgifter utgör grunden i datorseende.

1.1.1 Neurala nätverk

Sedan 2012 har utvecklingen inom bildanalys drivits av framstegen inom djupa neurala nätverk [8]. Under tävlingen *ImageNet Large Scale Visual Recognition Challenge* presenterades modellen AlexNet av Alex Krizhevsky tillsammans med Ilya Sutskever och Geoffrey Hinton vilken uppnådde avsevärt bättre resultat än alla tidigare metoder. Modellen visade hur djupa neurala nätverk kan användas för klassificering och sedan dess har metoden varit standarden inom bildbehandling.

En central begränsning för djupinlärningsmetoder är att de kräver stora mängder annoterade träningsdata. Alltså, en databas med bilder vilka en modell kan tränas på. För detta arbete hade det alltså krävts en databas med bilder i vilka detaljer såsom spisar, väggar, fönster, dörrar med mera var markerade och annoterade. I en studie från 2025 där man ämnade att från magnetröntgenbilder känna igen tumörer användes ett 5-lager djupt Konvolutionellt neuralt nätverk (CNN) som tränades på ett dataset om 198 bilder innehållande båda klasserna fick man prestandaindikatorer som precision på 98.75% och recall 99.20% [10]. Det krävs alltså annoterade objekt i storleksordningen hundratals för att med den senaste tekniken få en modell med bra prestanda.

Trots de starka resultaten från djupa neurala nätverk är deras beroende av stora mängder annoterad träningsdata en betydande begränsning i praktiska tillämpningar. I fallet med byggnadsritningar är framtagandet av sådant dataset särskilt resurskrävande, då annotering kräver expertkunskap och noggrann manuell genomgång av en stor mängd ritningar. Detta gör att datadrivna metoder inte är ett realistiskt första steg i automatisering av bedömningsprocessen.

Mot denna bakgrund finns ett behov av metoder som kan fungera med en liten mängd träningsdata. Klassiska bildbehandlingsmetoder utgör här ett relevant alternativ, då de istället bygger på explicita matematiska jämförelser mellan strukturer och former i bilder. Sådana metoder kan, utan föregående träning, användas för att identifiera likheter mellan objekt baserat på deras geometriska egenskaper.

1.1.2 Klassiska modeller

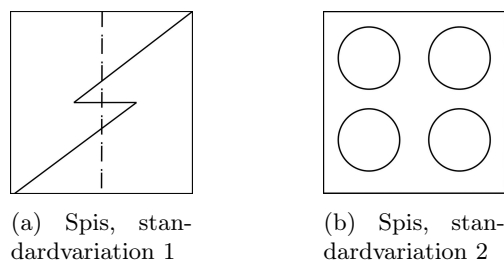
En central idé inom dessa metoder är att de bygger på att representera bilder som punktmängder, vilket möjliggör jämförelse mellan former. Ett mått som används i detta sammanhang är Hausdorff-avståndet, vilket kvantifierar hur väl två punktmängder överensstämmer med varandra [6]. Genom att jämföra en känd referensbild med delar av en bild kan potentiella matchningar identifieras. En annan metod som används i detta syfte är Template Matching vilken även den jämför en referensbilds punktmängd med delmängder i en annan bild [9].

Trots sina fördelar har klassiska metoder sina begränsningar. De är känsliga för brus, rotationer, skalning etc. vilket är särskilt relevant i byggnadsritningar då det inte finns tydliga standarder och utformning, skala och andra relevanta aspekter kan variera med arkitekt, ritstandard eller datorprogram. För att uppnå robusta resultat krävs alltså att metoderna kompletteras med förbehandling och anpassningar som tar hänsyn till variationerna.

1.2 Syfte

Automatisk identifiering av objekt i byggnadsritningar kommer med utmaningar ur ett tekniskt perspektiv. För att skapa ett ramverk för senare arbeten riktar detta arbete in sig på att lokalisera spisar i planritningar från Göteborgs kommuns bygglovshandlingar under perioden 2022-2025. Spisar valdes eftersom de är vanligt förekommande, det finns en stark korrelation mellan antalet spisar och antalet lägenheter i en planritning samt att de har tydliga mer varierade symbolrepresentationer. Detta gör dem lämpliga som ett första testobjekt för att undersöka robustheten hos olika metoder för objekt-detektion.

Ur handlingarna kunde man utläsa att det finns två olika standarder som används för att beteckna en spis i ritningar, vilka visas i Figur 1. Trots denna begränsning i antalet symboltyper förekommer variationer inom de två kategorierna. De har olika rotation och linjetjocklek, det förekommer bokstäver som markerar att det är en ugn i samband med spisen och det förekommer även markeringar för skåp över spisen. Dessa faktorer innebär, trots att det finns ett begränsat antal klasser, att uppgiften inte är trivial då varje klass innehåller en stor mängd underklasser.



Figur 1: Exempel på två standardvarianter av spisar som hittats i bygglovshandlingar i Göteborgs Kommun

Mot bakgrund av den begränsade tillgången av annoterad träningsdata är syftet med arbetet att undersöka i vilken utsträckning klassiska bildbehandlingsmetoder kan användas för att identifiera dessa objekt. I kontrast till dataintensiva metoder, såsom djupa neurala nätverk, bygger dessa på matematiska jämförelser och kräver ingen träningsfas. Detta gör dem intressanta i sammanhang som dessa, där dataset behöver skapas iterativt och där man vill undvika att lägga stora resurser på annotering.

Särskilt studeras de två metoderna Hausdorff-avstånd och Template Matching, samt olika varianter och implementationer av dessa. Målet är att utvärdera deras potential att fungera som ett första steg i en automatiserad bedömningsprocess eller som stöd för att generera annoterad data för djupa neurala nätverk. Resultaten kan även ses som stöd för framtida arbete där mer avancerade maskininlärningsmetoder kan användas på en preliminärt annoterad datamängd.

2 Teori

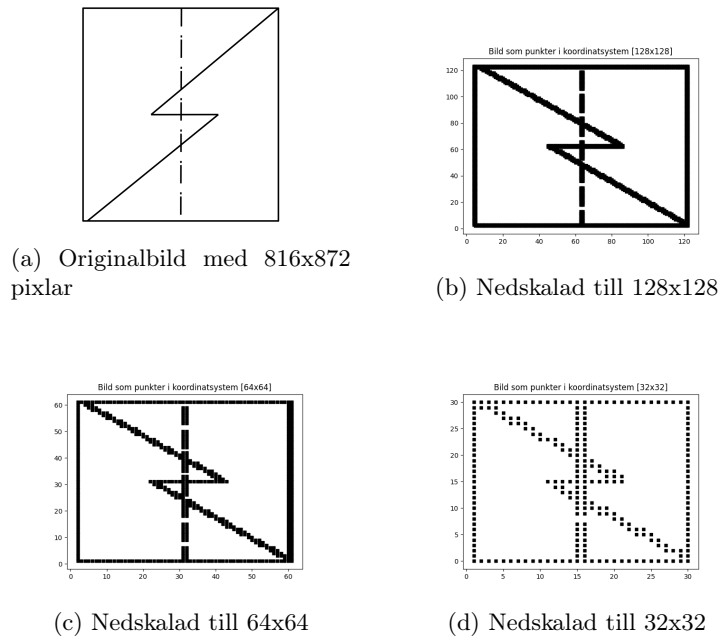
I detta avsnitt framförs en presentation av teorin som ligger till grund för de metoder som används i arbetet. Syftet är att skapa en grundförståelse för metodernas förutsättning, tillämpningsområde och begränsningar.

2.1 Digital representation av en bild

Antag att vi har en digital svartvit bild I med dimensionerna $M \times N$. Varje pixel har ett värde $I(x, y) \in \{0, 1\}$, där 1 representerar svart och 0 representerar vit. Bildens punktmängd A kan då definieras som:

$$A = \{(x_i, y_j) : (x_i, y_j) \in I(x, y); i = 1, \dots, M; j = 1, \dots, N\}$$

I Figur 2 visas en bild och dess punktmängd representerad i ett x-y koordinatsystem nedskalat till olika nivåer.



Figur 2: Originalbild (a) och dess punktmängdsrepresentationer i olika nedskalningar (b)-(d)

När en kontinuerlig bild, som den i Figur 2a, transformeras till en punktmängd A sker en diskretisering av dess pixlar. Kvaliteten på den diskretiserade bilden är direkt beroende av dess dimension $M \times N$ i förhållande till originalbilden. En reducering i dimension leder till att vi samplar funktionen $I(x, y)$ mer sällan och därav kan punktmängden A inte längre representera detaljer från den kontinuerliga bilden. Vidare noteras att bilden är spegelvänd i den diskretiserade representationen. Detta saknar dock praktisk betydelse, eftersom orienteringen inte påverkar punktmängdens struktur eller de egenskaper som analyseras.

2.2 Inter Area Interpolation

Givet en ursprungsbild $I(x, y)$ med storlek $w \times h$ definieras vid skalning av bilden de nya pixlarna som $I'(i, j)$ med storlek $w' \times h'$. Vid nedskalning från dimensioner $x \times y$ till $i \times j$ definieras skalfaktorer som

$$s_x = \frac{w}{w'}, \quad s_y = \frac{h}{h'}.$$

Varje pixel (i, j) motsvarar då ett rektangulärt område i ursprungsbilden

$$R_{i,j} = \{(x_i, y_j) : x_i \in [is_x, (i+1)s_x], y_j \in [js_y, (j+1)s_y]\}$$

där

$$i = 0, \dots, w' - 1, j = 0, \dots, h' - 1.$$

Pixelvärdet i den nya bilden definieras som en viktad summa av bidrag från alla ingångspixlar (x, y) vars area överlappar $R_{i,j}$

$$I'(i, j) = \sum_{x,y} w_{i,j}(x, y) I(x, y) \quad (1)$$

där vikten $w_{i,j}(x, y)$ ges av den normerade överlappningsarean

$$w_{i,j}(x, y) = \frac{\text{Area}(R_{i,j} \cap P_{x,y})}{\text{Area}(R_{i,j})}.$$

2.3 Hausdorff-avstånd

I en svartvit bild $I(x, y)$ med dimensionerna $M \times N$ har varje pixel ett värde $I(x, y) \in \{0, 1\}$, där 1 representerar svart och 0 representerar vitt. Bildens punktmängd $\mathcal{A}$ kan då definieras som

$$\mathcal{A} = \{(x, y) \in \{1, \dots, M\} \times \{1, \dots, N\} \mid I(x, y) = 1\}.$$

Givet de två punkterna $A = (a_1, \dots, a_M)$ och $B = (b_1, \dots, b_N)$ är Hausdorff avståndet mellan de två definierat som

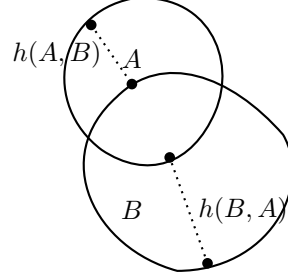
$$H(A, B) = \max(h(A, B), h(B, A)) \quad (2)$$

där

$$h(A, B) = \max_{a \in A} \min_{b \in B} \|a - b\|,$$

$$h(B, A) = \max_{b \in B} \min_{a \in A} \|b - a\|.$$

I Figur 3 följer en visuell representation av det oriktade Hausdorff-avstånd för två mängder A och B . Avstånden $h(A, B)$ och $h(B, A)$ representerar det största avståndet från en punkt i A till närmsta punkt i B och vice versa. Hausdorff-avståndet $H(A, B)$ tar hänsyn till båda riktningarna och mäter därmed hur långt mängderna ligger ifrån varandra i värsta fall.



Figur 3: Visuell representation av det riktade Hausdorff-avståndet mellan två mängder A och B

2.4 Modifierat Hausdorff-avstånd

En begränsning Hausdorff-avstånd är dess känslighet för brus och extremvärden eftersom en felaktig eller brusig pixel kan dominera beräkningen [4]. För att minska känsligheten för brus och extremvärden används istället metoden Modifierat Hausdorff-avstånd föreslagen av M.-P Dubuisson och A.K. Jain vid jämförelse av 24 olika versioner av Hausdorff-avstånd för objektigenkänning. Givet de två punktmängderna A, B ges det största medelvärde utav avstånden mellan $(a_1, \dots, a_M)$ och punktmängden B samt $(b_1, \dots, b_N)$ och punktmängden A enligt

$$H(A, B) = \max\left(\frac{1}{M} \sum_{i=1}^M d(a_i, B), \frac{1}{N} \sum_{j=1}^N d(b_j, A)\right). \quad (3)$$

där avstånden ges av

$$d(a_i, B) = \min_{b \in B} \|a_i - b\|,$$

$$d(b_i, A) = \min_{a \in A} \|b_i - a\|.$$

2.5 Singularvärdesuppdelning (SVD)

SVD är en metod för faktorisering av matriser som används bland annat för bildkomprimering [1]. Metoden bygger på att identifiera de viktigaste riktningarna i datan och hur starkt dessa är korrelerade till ursprungsmatrisen, vilket beskrivs av de så kallade singulära värdena. Genom att ordna de singulära värdena efter storlek kan man approximera matrisen med lägre rang och därmed behålla de viktigaste detaljerna. För att beskriva detta ytterligare kan vi uttrycka SVD-formeln för en matris $\mathbf{A}$ och faktoriseringen till dess grundläggande komponenter.

Antag att vi har en matris $\mathbf{A}$ med storlek $m \times n$ där $m \geq n$. Då är $\mathbf{A} = U\Sigma V^T$, där U har storleken $m \times m$ och uppfyller $U^T U = I$ (identitetsmatrisen), V har storleken $n \times n$ och uppfyller $V^T V = I$, och $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_n)$, där $\sigma_1 \geq \dots \geq \sigma_n \geq 0$.

$$\mathbf{A} = \underbrace{\begin{bmatrix} | & | & & | \\ u_1 & u_2 & \cdots & u_m \\ | & | & & | \end{bmatrix}}_U \underbrace{\begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_n \end{bmatrix}}_{\Sigma} \underbrace{\begin{bmatrix} v_1 & v_2 & \cdots & v_n \end{bmatrix}^T}_V$$

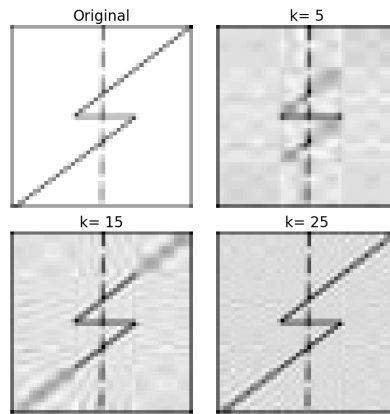
Detta innebär att en bild som representeras av matrisen $\mathbf{A}$, kan approximeras med matrisen $\mathbf{A}_k$ som innehåller ett begränsat antal komponenter k enligt

$$\mathbf{A} \approx \mathbf{A}_k = \sum_{i=1}^k u_i \sigma_i v_i^T \quad (4)$$

där u_i beskriver variation i vertikal riktning, v_i^T beskriver rotation i horisontell riktning och σ_i är en viktning för vardera komponent. Vikterna, även kallat de singulära värdena, är kvadratrötterna av egenvärdena av matrisen $\mathbf{A}^T \mathbf{A}$ i enlighet med nedanstående ekvation:

$$\begin{aligned} \det(\mathbf{A}^T \mathbf{A} - \lambda \mathbf{I}) &= 0 \\ \sigma_i &= \sqrt{\lambda_i}. \end{aligned} \quad (5)$$

För att visualisera detta visar Figur 4 originalbilden samt tre låg-rank approximationer med SVD. Man kan tydligt se hur approximationen gradvis närmar sig originalbilden med ökat k .

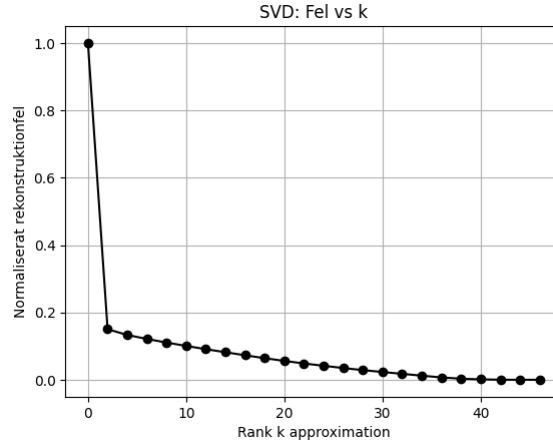


Figur 4: Originalbild med tre approximationer av rank- k

Approximationens fel, eller komprimeringsfelet av bilden/matrisen $\mathbf{A}$ kan kvantifieras genom Frobeniusnormen ($\mathbf{e}_F$). Frobeniusnormen mäter storleken på skillnaden mellan originalmatrisen och dess rank- k approximation, vilket ger ett mått på hur mycket information som går förlorad vid SVD-komprimering. Det relativa approximationsfelet definieras som:

$$\mathbf{e}_F = \frac{\|\mathbf{A} - \mathbf{A}_k\|_F}{\|\mathbf{A}\|_F}. \quad (6)$$

där $\mathbf{A}_k$ är rank- k approximationen av originalmatrisen $\mathbf{A}$. Detta ger ett numeriskt värde på approximationsfelet oberoende av originalmatrisens absoluta storlek vilket visualiseras i Figur 5. Det numeriska värdet på approximationsfelet beroende av rank- k ger en förståelse för hur väl approximationen bevarar de viktigaste detaljerna i bilden.



Figur 5: Frobeniusnorm av fel per approximation av rank k

2.6 Rotation av bilder i två dimensioner

En bild representerad av matrisen $\mathbf{A} \in \mathbb{R}^{M \times N}$ kan roteras med tvådimensionell Euclidisk transformation [16]

$$\mathbf{A}' = \mathbf{R}\bar{\mathbf{A}}.$$

$\mathbf{R}$ är en ortonormal rotationsmatris och uppfyller därmed följande villkor: $\mathbf{R}\mathbf{R}^T = \mathbf{I}$ och $\|\mathbf{R}\| = 1$. För en rotation med vinkeln θ ges rotationsmatrisen $\mathbf{R}$ av

$$\mathbf{R} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}. \quad (7)$$

Vi transformerar alltså varje koordinat (x, y) i $\mathbf{A}$ till en ny koordinat (x', y') enligt

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \underbrace{\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}}_{\mathbf{R}} \cdot \underbrace{\begin{bmatrix} x \\ y \end{bmatrix}}_{\mathbf{u}}. \quad (8)$$

Om vi tar två punkter $(x_1, y_1) = \mathbf{u}$ och $(x_2, y_2) = \mathbf{v}$ innan rotation så blir dess euklidiska avstånd

$$d = \|\mathbf{v} - \mathbf{u}\| = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}.$$

Efter rotation har vi punkterna $(x'_1, y'_1) = \mathbf{R}\mathbf{u}$ och $(x'_2, y'_2) = \mathbf{R}\mathbf{v}$. Eftersom $\mathbf{R}$ är ortonormal gäller att

$$\|\mathbf{R}(\mathbf{v} - \mathbf{u})\| = \|\mathbf{v} - \mathbf{u}\| \text{ och } (\mathbf{R}\mathbf{u}) \cdot (\mathbf{R}\mathbf{v}) = \mathbf{u} \cdot \mathbf{v}, \quad (9)$$

vilket innebär att både avståndet mellan punkter och vinkeln mellan vektorer bevaras i bilden, alltså förvrängs inte bildens geometriska egenskaper efter rotation.

2.7 Template matching

Template matching är en objektidentifieringsmetod. Metoden går ut på att en referensbild T jämförs med en bild på alla möjliga positioner (x, y) . I detta arbete används `TM_COEFF_NORMED` implementerad i bildbehandlingsbiblioteket `OpenCV` [13].

Låt $T(x', y')$ beteckna referensbilden med dimensioner $m \times n$ och $I(x, y)$ bilden som undersöks. För varje position (x, y) extraheras ett bildutdrag $I(x + x', y + y')$ av samma dimensioner som referensbilden. För att eliminera effekten av ljusförskjutning definieras de medelvärdesbildade funktionerna

$$\begin{aligned} T'(x', y') &= T(x', y') - \mu_T, \\ I'(x + x', y + y') &= I(x + x', y + y') - \mu_I \end{aligned}$$

där

$$\begin{aligned} \mu_T &= \frac{1}{mn} \sum_{x', y'} T(x', y'), \\ \mu_I &= \frac{1}{mn} \sum_{x', y'} I(x + x', y + y') \end{aligned}$$

är medelvärdena för referensbilden respektive bildutdraget. Likhetsmättet mellan referensbilden och utdraget definieras därefter som den normaliserade korskorrelationen mellan de båda

$$R(x, y) = \frac{\sum_{x', y'} (T'(x', y') \cdot I'(x + x', y + y'))}{\sqrt{\sum_{x', y'} T'(x', y')^2 \cdot \sum_{x', y'} I'(x + x', y + y')^2}}. \quad (10)$$

Vi noterar vidare att $R(x, y) \in [-1, 1]$ och att $R(x, y)$ är likställt med cosinus av vinkeln mellan de två vektorer som erhålls genom att vektorisera T' och I' (se Sats 2 i Appendix A för ett bevis). Det högst korrelerade området mellan T och I ges utav $\max R(x, y) \forall (x, y)$.

3 Metod

Detta kapitel beskriver tillvägagångssättet som användes för att utveckla och utvärdera de analyser som föreslås i arbetet. Innan arbetet genomfördes bestämdes metoderna som skulle användas genom informationssökning, matematisk analys och ett antal strategiska tester.

3.1 Implementation

Undersökningen genomfördes genom att implementera de valda metoderna för att sedan analysera träffsäkerheten och tidskomplexitet. Eftersom syftet var att jämföra objektlokaliseringmetoder som kräver små datamängder skapades en ram för implementationen där metoderna kunde jämföras på ett likvärdigt sätt. Kärnan i objektlokalisering är att givet en mängd $\mathbf{A}$ och ett referensobjekt r , hitta alla $\{a \in \mathbf{A} \mid a \sim r\}$. I detta sammanhang är prestationsmått som precision och recall mindre lämpliga, eftersom $\{a \in \mathbf{A} \mid a \not\sim r\} \gg \{a \in \mathbf{A} \mid a \sim r\}$, vilket exempelvis leder till ett stort antal True Negatives i förhållande till antalet True Positives. *Träffsäkerheten* definieras som antalet korrekta träffar dividerat på det totala antalet prediktioner och analys av algoritmens tidskomplexitet gjordes i termer av Big O-notation.

3.1.1 Svepande fönster

För att lokalisera ett objekt i en bild krävs en metod för att söka igenom ritningen för att kunna mäta skillnader och likheter med referensobjektet. En vanlig teknik för detta är att använda en svepande fönster funktion, där ett fönster med förutbestämda dimensioner förflyttas över hela bilden stegvis. Vid varje position extraheras bilddata för att med hjälp av vald algoritm göra beräkningar på objektets likhet.

Metoden är i detta arbete lämplig eftersom att spisar kan förekomma på olika positioner i ritningar. Genom att använda ett svepande fönster säkerställs att ritningen genomsöks utan att göra antaganden om spisens position. Den ger även möjligheten att söka efter samma objekt med olika storlekar.

Enligt tester som gjordes på en delmängd i datasetet varierade storlekarna på spisen mellan 30-70 pixlar vilket motiverade valet att använda flera olika fönster storlekar, alltså repetera svepande fönsteralgoritmen för flera storlekar på fönster. Känslighet mot brus och risken att lokalisera falska positiva exempel motiverade valet att använda flera storlekar på fönster.

Nackdelen med svepande fönster är dess beräkningskostnad, särskilt vid små stegstorlekar och flera skalor [7, p. 156-157]. Trots detta valdes denna tekniken då den är relativt enkel att implementera, generell och väl lämpad för att jämföra olika algoritmer under samma förutsättningar.

3.1.2 Non Maximum Supression

En prestationsförbättrande åtgärd som gjordes är Non-Maximum Suppression (NMS). Eftersom fönstret förflyttar sig med ett förutbestämt avstånd kommer det leda till att ett stort antal detektioner hamnar inom samma område på grund av den låga steglängden och de olika fönsterstorlekarna. NMS är generellt sett en svag algoritm vid objekt-detektering av komplexa bakgrunder och överlappande objekt på grund av att den använder en greedy strategi vid urvalet av fönsterstorlek [17]. I detta arbete är ritningarna av sådan karaktär att bakgrundsdetektioner, överlappande klasser och komplexa strukturer inte är något som behöver beaktas, varför NMS är särskilt tillämplig. Dessutom minskar brus vid en konstruktion av en klassificeringsalgoritm eftersom det totala antalet potentiella detektioner minskar, vilket kan minska risken för till exempel överanpassning. I Figur 6 visas en illustration av hur NMS fungerar inom objektlokalisering där flera detektioner reduceras ned till en slutgiltig kandidat inom området.



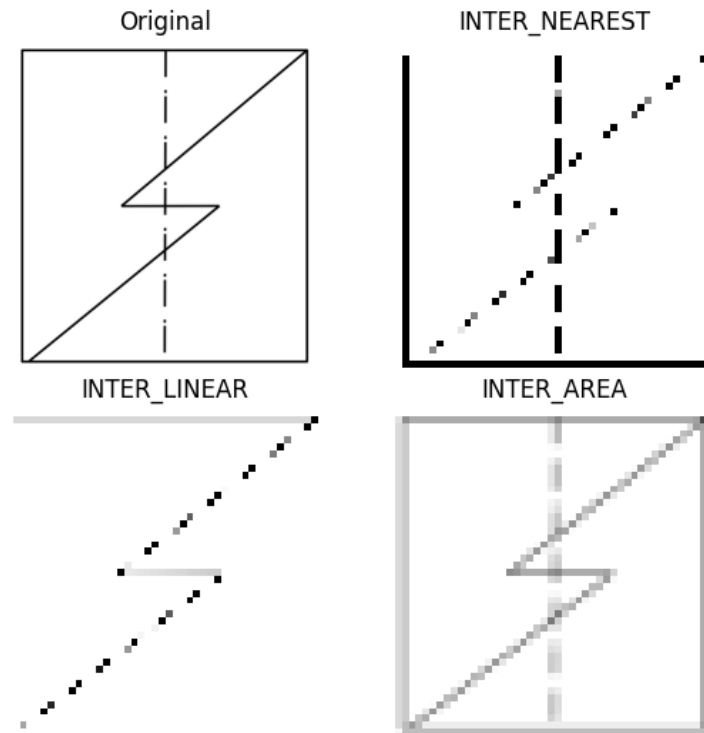
Figur 6: Non-Maximum Supression för objektlokalisering

3.1.3 Interpolation

Som tidigare nämnt användes olika fönsterstorlekar i svepande fönster algoritmen. Eftersom att referensbilderna har en fast storlek uppstår ett behov av att anpassa referensbilden utefter de extraherade bildutdragen innan vidare bearbetning. Vid bildskalning används interpolationsmetoder för att approximerar intensitetsvärden i en diskret bild baserat på en ursprungsbild. I detta arbete användes `INTER_AREA` från bildbehandlingsbiblioteket `OpenCV` [12] föreslagen i en empirisk studie gjord på olika maskininlärningsbaserade objektigenkänningsalgoritmer [5].

I denna metod representerar varje pixel i den nedskalade bilden ett genomsnittligt intensitetsvärde över ett motsvarande delområde i ursprungsbilden. Detta motiverar varför metoden är lämplig just för nedskalning av bilder. När en bild skapas ned minskar antalet pixlar, vilket innebär att ursprungliga pixlar måste sammanfattas i en enda ny pixel. Genom att beräkna ett medelvärde över motsvarande område i originalbilden bevaras den samlade strukturen på ett mer representativt sätt.

I Figur 7 ser vi tre olika interpolationsmetoder från `OpenCV` bredvid varandra i vilken noteras att `INTER_AREA` bäst bevarar detaljer i bilden.

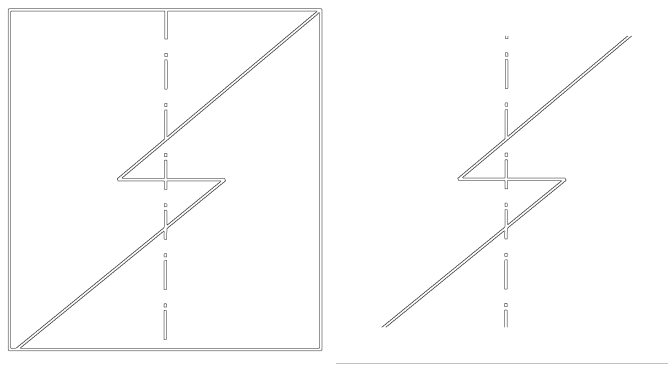


Figur 7: Originalbild med dimensioner 816×872 nedskalad till 48×48 med tre olika interpolationsmetoder

3.1.4 Negativ Padding

I våran data är spisar markerade som en kvadrat innehållande en blix-liknande symbol. Det är just denna blix symbol som är mest utmärkande för en spis och inte kvadraten runt den. Våra referensbilder består dock av både kvadrat och symbol. Flera andra objekt på ritningar består också av en kvadrat med en symbol. När vi jämför spis referensbilden med en av dessa andra objekt kan det ge ett falskt positivt resultat. Detta kan ske eftersom självaste symbolen är en procentuellt mindre del av bildens pixlar, jämfört med kvadraten runtom.

För att försöka lösa detta problem kan man helt enkelt ta bort kanterna i referensbilderna. Med hjälp av kod kunde vi ta bort 10% av referensbilderna längs alla kanter. Alltså blev en referensbild med 100×100 pixlar effektivt en bild med 80×80 pixlar, alla svarta pixlar mindre än 10 pixlar från bildens kant blir vita. Detta tar effektivt bort kvadraten runt alla våra referensbilder som kan ses på exemplet i figur 8.



Figur 8: Referensbild (vänster), och referensbild med 10% negativ padding (höger).

3.1.5 Normering

Normering gör att det blir mer rättvist vid uträkning av Hausdorff-avstånd. Eftersom Hausdorff-avstånd är ett avståndsmått har det en bias mot bilder med lägre dimensioner. När man använder olika fönsterstorlekar, har de större fönstren naturligt längre avstånd och kan därför leda till missvisande resultat. Hausdorff-avståndet normeras med fönstrets diagonallängd för att få ett avstånd som kan jämföras med alla olika fönsterstorlekar.

För template matchning användes funktionerna `TM_CCORR_NORM` och `TM_CCORR` från `OpenCV` biblioteket. Där den förstnämnda normerar intensiteten på pixlarna i både fönstret och referensbilden genom att subtrahera deras respektive medelvärde. Utan normering ger en spis ritad med hög kontrast ett högre värde än en spis ritad med lägre kontrast. Normering av template matchning gör att intensiteten av pixlarna inte påverkar resultaten, det är istället endast mönstret som pixlarna bildar.

3.1.6 Klassificering

För att möjliggöra en jämförbar utvärdering av de undersökta algoritmerna gjordes en förenkling av klassificeringsproblemet. Istället för att identifiera spisar i hela planritningar avgränsades datasetet till ett antal beskurna utdrag ur planritningar där varje utdrag innehöll exakt en spis. För att säkerställa en representativ bedömning valdes utdrag där spisar hade olika rotation, utformning och skärpa.

Denna avgränsning förenklar problemet genom att eliminera behovet av en mer avancerad klassificeringsalgoritm vilken kan hantera flera sant positiva i en och samma bild. Klassificeringen baserades därför enbart på vilken kombination av fönsterstorlek och position som gav lägst Hausdorff-avstånd eller bäst värde vid Template Matching.

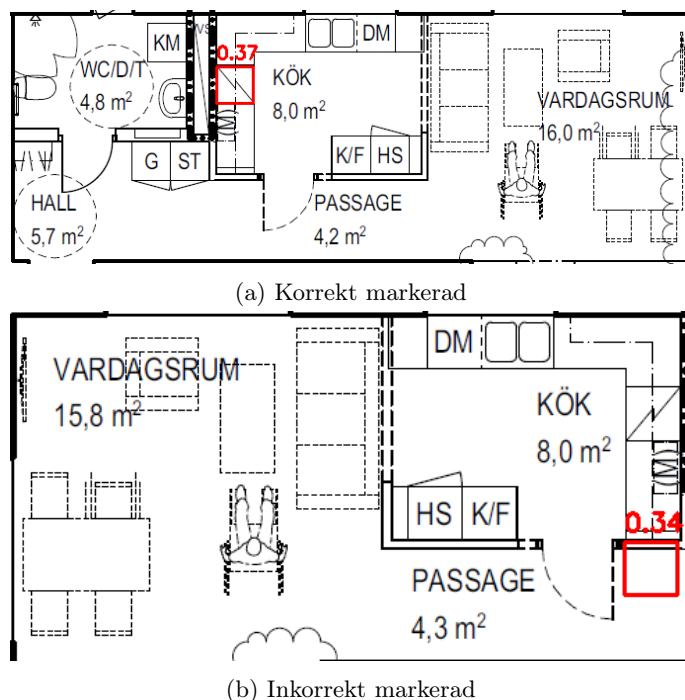
3.2 Data

I arbetet användes bygglovshandlingar för flerfamiljshus från Göteborgs stad under perioden 2022-2025. Totalt består det utav 122 objekt, eller unika fastigheter. Varje objekt innehåller flertalet handlingar såsom markplaner, VA-ritningar, nämndbeslut samt planritningar. Vi var endast intresserade av planritningarna och behövde därmed manuellt filtrera bort allt annat.

Ritningarna var givna som vektorbilder i PDF format. Våra metoder kräver att ritningarna är pixelrepresenterade, till exempel i PNG format. För att konvertera från PDF till PNG användes Python biblioteket `pdf2image` med 100 DPI. Dessa png bilder behövdes skäras ner eftersom de fullstora ritningarna var mycket stora och hade för många objekt som går att se i exempelritningen i appendix C. Därför beskars bilderna för att innehålla en spis per bild. Planritningar över flerbostadshus innehåller många lägenheter som ser ut på ungefär samma sätt. Detta leder till ett stort antal dubletter vilka filtrerades ut ur vårt dataset för att undvika att artificiellt stärka eller minska resultaten. Se appendix C för exempel av planritning.

3.3 Test av metoder

För att testa metodernas precision användes de på ett dataset av 103 stycken mindre bilder. Testbilderna är delar av de större ritningarna och innehåller exakt en spis. Metoderna användes för att markera det område som gav bäst värde. De ritningar som hade lägst avstånd sparades sedan i en mapp som innehåller alla ritningar med markerade spisar. Resultaten kontrollerades sedan manuellt genom att räkna hur många testbilder som har en korrekt markerad spis. Figur 9 visar exempel på testbilder där Template matchning metoden markerat området med högst värde. Hausdorff, modifierad Hausdorff och normerad Hausdorff kördes sekventiellt i samma kod med och utan patchar. Varje metod hade sin egna tidsräknare, så tidsutvärdering är möjlig för respektive metod.



Figur 9: Exempel på korrekt och inkorrekt markerad spis med template matchning. Den röda rutan är det område med högst värde.

4 Resultat

De sammanställda resultaten erhållna av de olika metoderna presenteras i detta avsnitt. För att skapa en jämn grund för bedömning av de olika metoderna användes som tidigare nämnt samma dataset om 103 bilder för varje metod. Som referensbilder användes tre olika varianter på spisar vilka alla användes i svepande fönster algoritmen. Även för referensbild användes samma dataset genom hela arbetet. Genom att använda samma dataset i sin helhet minimeras risken att införa bias mot vissa metoder.

4.1 Numeriska resultat

Tabell 3 visar resultaten för de olika metoderna som användes utan negativ padding. Här kan vi se att normerad template matchning har högst träffsäkerhet, samtidigt som den metoden tog näst kortast tid att köra. Vi ser även en anmärkningsvärd skillnad i tid mellan Hausdorff-uträkningar och template matchning, där Hausdorff-metoderna är ungefär 7.5 gånger långsammare. Tabell 4 visar samma metoder, men med negativ padding. Det innebär att 10% av kanterna i referensbilden har skalats av och därför ser vi att koden är mycket snabbare.

De olika metoderna genomfördes och testades enligt 3.3.

Tabell 3: Resultat utan negativ padding

Metod	Träffsäkerhet [%]	Tid [s]
<i>Hausdorff</i>	4.9	9024
<i>Modifierad Hausdorff</i>	22.3	3031
<i>Normerad modifierad Hausdorff</i>	29.1	2959
<i>Template matchning</i>	9.7	396
<i>Template matchning - Normerad</i>	34.0	507

Tabell 4: Resultat med negativ padding

Metod	Träffsäkerhet [%]	Tid [s]
<i>Hausdorff</i>	8.7	5038
<i>Modifierad Hausdorff</i>	13.6	1704
<i>Normerad modifierad Hausdorff</i>	19.4	1766
<i>Template matchning</i>	8.7	312
<i>Template matchning - Normerad</i>	35.9	457

4.2 Tolkning av resultat

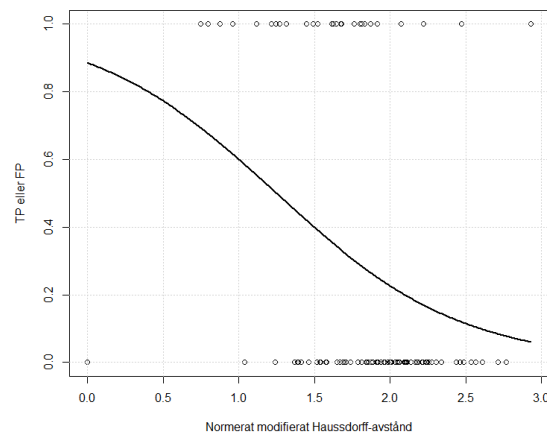
Resultatet för de metoder med högst träffsäkerhet i Tabell 3 och Tabell 4 analyserades vidare. Given analysmetods värde vid högsta matchningen hämtades och för varje klassificering i angavs ett värde $y_i \in \{0, 1\}$ där $y_i = 1$ anger korrekt klassificering och $y_i = 0$ anger felaktig klassificering.

I tabell 3 ser vi att Hausdorff har betydligt sämre resultat än modifierad Hausdorff. Modifierat Hausdorff-avstånd använder istället ett medelvärdesbildat avståndsmått mellan två mängder, vilket innebär att metoden till skillnad från Hausdorff-avstånd har ett inbyggt filter mot extrempunkter och brus. I en verklig miljö spelar faktorer som dessa en stor roll i algoritmernas träffsäkerhet då skalning, rotation, brus mer flera faktorer spelar in. Vi ser även att den normerade metoden har bättre resultat än utan. Detta är rimligt då alla avstånd skalas om till ett enhetsavstånd. Eftersom icke normerade algoritmer riskerar att skapa en bias där större bilder har generellt större avstånd och sorteras bort. Vi kan även konstatera att normerad template matchning har överlag bäst resultat, både när det gäller träffsäkerhet och tidsåtgång.

I tabell 4 har vi tagit med negativ padding som ett förarbete innan beräkningarna sker. Vi skalar bort 10% av kanterna från både referensbilden och ritningen. Detta skulle kunna leda till att vi får med mindre brus när uträkningarna ska göras. Vi ser istället att resultaten har sjunkit för modifierat och normerat modifierat Hausdorff-avstånd. Vid negativ padding med 10 % försvinner effektivt 36% av bildens totala yta. För medelvärdesbildning innebär detta att medelvärdet beräknas på en reducerad yta. Alltså blir medelvärdet mindre tillförlitligt eftersom färre datapunkter ingår. Hausdorff-avstånd visar på en ökad träffsäkerhet med 77.5%. Detta beror troligen på att extrempunkter som vi kan räkna som brus filtreras bort i ritningen och gör spisen mer isolerad. Template matchnings största svaghet ligger inte i punkter som kan agera som brus, därför ser vi ingen större skillnad i varken vanlig template matchning eller normerad template matchning.

4.2.1 Normerat modifierat Hausdorff-avstånd

I figur 10 ser vi resultatet visualiserat i en graf. Den visar sambandet mellan det normerade modifierade Hausdorff-avståndet och det binära resultatet. Kurvan är en logistisk regressionskurva vilken beskriver sannolikheten för en korrekt markerad spis vid ett specifikt avstånd. Sannolikheten för en korrekt träff är högre när avståndet är lågt. Detta är rimligt då lågt avstånd betyder att bilderna är lika. Vi ser att det inte finns något tydligt tröskelvärde där vi kan separera på korrekta och felaktiga klassificeringar. Därför blir det svårt att sätta ett tröskelvärde på avståndet där vi kan få ut mer träffsäkra resultat samt använda en klassificeringsmetod vilken kan hantera fler än en sant-positiv per bild.



Figur 10: Normerat Hausdorff-avstånd resultat vs Klassificeringsresultat

4.2.2 Normerad template matchning

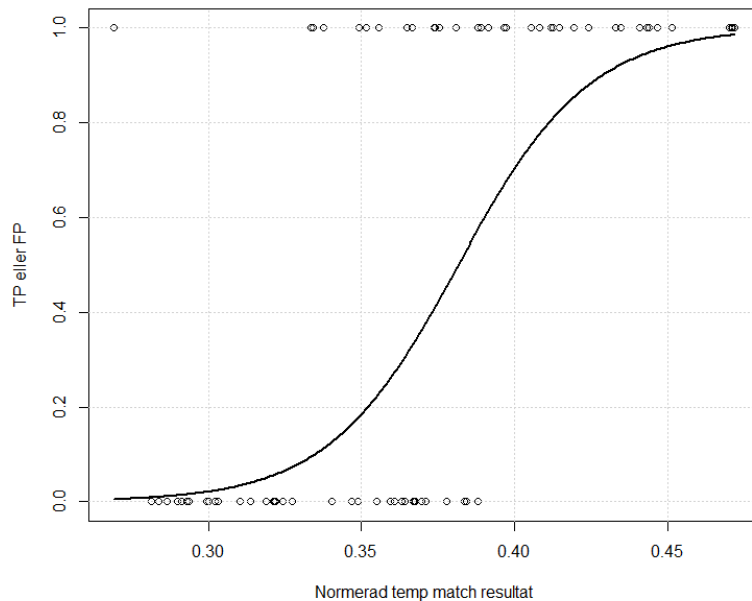
Av samtliga metoder uppvisade normerad template matchning med negativ padding högst träffsäkerhet. Genom att införa negativ padding ökade prestandan med 5.6% vilket anses som en väl konstruerad optimeringsmetod. Tidsåtgången för algoritmen vid införandet av negativ padding blev oförändrad vilket beror på att både jämförelsemetoden och negativ padding uppvisar ungefär linjär komplexitet. Givet en bild $I \in \mathbb{R}^{m \times n}$ har normerad template matchning tidskomplexiteten

$$O(nm)$$

vilket visas i Sats 3 och eftersom negativ padding förändrar bildens dimensioner med en konstant faktor c förblir den asymptotiska komplexiteten oförändrad

$$O((n - c)m) = O(nm - mc) = O(nm). \quad (11)$$

I figur 11 har vi på samma sätt som innan plottat en logistisk regressionsgraf. Skillnaden för normerad template matchning är att vi har data som stödjer ett tydligt tröskelvärde vid 0.43. Där är sannolikheten för ett lyckat resultat ca 90 % enligt den logistiska regressionsmodellen. Detta tyder på att normerar template matchning kan i större utsträckning skilja på korrekta och felaktiga matchningar än de Hausdorff-baserad metoderna. Det sker en överlappning mellan korrekta och felaktiga matchningar vilket innebär att tröskelvärdet inte är perfekt. Trots det är metoden tillräckligt stabil för att kunna användas för att generera träningsdata till ett neuralt nätverk.



Figur 11: Normerad Template matchning vs Klassificeringsresultat

4.3 Kantdetektion som förbearbetningsmetod

För att använda Hausdorff metoden optimalt bör objekt man söker efter i bilden ha tydliga kanter som liknar referensbilden [6]. Om man endast konverterar bilden till binärt färgad kan man få en hel del brus. Istället kan man först använda en gaussisk utjämning på bilden och sedan använda en kantdetektionsalgoritm. Kantdetektionsalgoritmer kan ta fram tydligare kanter runt objekt i en bild, vilket är precis vad vi vill ha för Hausdorff metoden.

Vi provade att använda Canny algoritmen [3] på ritningarna innan Hausdorff metoden användes. Detta gav dock sämre resultat med sämre träffsäkerhet som kan ses i figur 12 och 13 i appendix D.1. De sämre resultaten kan bero på att ritningarna redan har relativt tydliga kanter. Den gaussiska utjämning suddar då ut de annars tydliga kanterna och introducerar brus istället för att jämna bort det. Kantdetektionsalgoritmen kan sedan göra detta brus till falska kanter som stör bildigenkänningsmetoden.

4.4 Resultat CNN

Vidare har konvolutionella neurala nätverk (CNN) även utvärderats som metod. Eftersom momentet främst syftar till att fungera som en förstudie inför det slutliga målet, har analysen inte genomförts med samma grad av teoretiskt fördjupning som tidigare metoder.

4.4.1 Bakgrund

På grund av den höga tidsåtgången och den relativt osäkra träffsäkerheten hos Hausdorff-avstånd och template matchning, har vi testat att träna ett neuralt nätverk. Vi har implementerat modellen yolo8s med hjälp av ultralytics. Vi har markerat objekt i 76 ritningar manuellt, där det förekommer flera lägenheter i samma ritning. Totalt hade vi 203 spisar som användes för träning och validering. För att kunna utföra träning har NVIDIAs program CUDA [11] använts för att kunna använda grafikkort som utför programmet.

4.4.2 Resultat

I figur 14 ser vi hur det neurala nätverket har identifierat spisen till en sannolikhet på 0.67. Vid vidare analys är det viktigt att hitta ett tröskelvärde vilket ger en balans mellan olika utvärderingsmått. I figur 15 ser vi hur modellen har identifierat samtliga spisar i ritningen.

5 Diskussion

I detta kapitel analyseras och utvärderas resultaten från kapitel 4. Metoderna jämförs i form av prestanda, begränsningar och praktisk användbarhet. Störst fokus är på träffsäkerhet och tidsåtgång men även varför vissa metoder presterar bättre än andra. Vidare diskuteras även alternativa metoder och begränsningar i datan.

5.1 Jämförelse av metoderna

I tabell 3 och 4 ser vi att tidsåtgången skiljer sig markant mellan de olika metoderna. Hausdorff metoderna beräknar avståndet mellan varje svart pixel i en bild och varje svart pixel i referensbilden. Antalet avståndsberäkningar ökar därmed kvadratisk med antalet pixlar i referensbilden (bevis i Appendix 1, Sats 1). Template matchning jämför istället varje pixel med exakt en annan pixel, antalet beräkningar ökar därmed linjärt med referensbildens storlek (bevis i appendix sats 3). Detta kan vara en anledning till att template matchnings metoderna exekverades snabbare än Hausdorff varianterna.

Träffsäkerheten för modifierat Hausdorff-avstånd var markant bättre Hausdorff-avstånd. Detta är rimligt eftersom att Hausdorff-avståndet är känsligt mot extremvärden. En pixel eller linje kan få stor påverkan på resultatet, även om fönstret ligger rätt på spisen. Eftersom modifierat Hausdorff-avstånd använder medelvärdesbildning kan extremvärden neutraliseras. En ytterligare förbättring av det modifierade Hausdorff-avståndet var normering. Normering leder till att varje fönsterstorlek får samma avståndsenhet vilket gör att de kan jämföras mer rättvist. Större fönsterstorlekar har naturligt längre avstånd och därför kan missvisande resultat uppstå när normering inte används.

Den metod som gav bäst resultat är normerad template matchning. Vi ser en ännu större fördel med att normera template matchning än för Hausdorff i resultaten. Detta kan bero på att template matchning använder sig av gråskalade bilder istället för binära. Utan att normera får då svagt ritade spisar lägre värde än något som är vagt likt en spis men ritad med starkare intensitet.

Negativ padding gav olika effekt beroende på metod. För Hausdorff-avstånd och normerad template matchning ser vi en ökad träffsäkerhet. För Hausdorff-avstånd kan detta bero på att en del extrempunkter från brus beskärs bort och för template matchning ser vi en liten skillnad, vilken kan bero på faktorer såsom datasetets och algoritmens egenskaper. För både icke-normerat och normerat modifierat Hausdorff-avstånd ser vi å andra sidan att träffsäkerheten har minskat. Detta kan bero på att färre punkter bidrar till medelvärdet, vilket leder till större risk för brus och därmed inkonsekventa resultat.

5.1.1 Begränsningar i data

Ritningarna i datasetet var delar av planritningar. På grund av husens varierande utformning hade även rum och således spisarna olika rotationer och skala. Som vi tidigare visat ställer de olika metoderna vissa krav på dessa egenskaper, vilket i slutändan påverkar deras robusthet. För vårt syfte, att hitta en spis, hade ritningarna mycket irrelevant information. Det kunde vara objektbeskrivningar, dimensioner på rum eller hyllor. Bruset som uppstod av detta kunde ge avvikelser som bidrog både till falsk negativa och falsk positiva träffar. Spisarnas symbol kunde också skilja sig mellan olika ritningar. Texten, hyllorna och faktumet att alla spissymboler varierade bidrog till en potentiellt försämrad robusthet.

5.2 Framtida metoder

En metod som används i många andra bildigenkännings sammanhang idag är ett konvolutionellt neuralt nätverk (CNN). Dessa neurala nätverk använder sig av fältningsoperationer på flerdimensionell data, exempelvis en bild [14].

Tidigt i arbetet upptäckte vi "You Only Look Once"(YOLO) [15] som är en färdigtränad CNN modell som kan tränas för att hitta och markera objekt i bilder, givet att man har markerad träningsdata. Vi provade att markera 76 olika ritningar med flera spisar och tränade sen YOLO modellen på den markerade datan. Den resulterande modellen kunde sedan markera alla spisar på alla bilder vi testade med.

Ett allmänt problem med neurala nätverk är att det är i princip omöjligt att veta precis hur de kommer fram till sitt resultat. De fungerar som en svart låda och det är svårt att utvärdera vad i metoden som eventuellt har gått fel. Det kommer att krävas ytterligare tester, helst på ritningar som inte har använts vid träning och validering.

5.2.1 SVD som förbearbetningsmetod

I framtida arbeten skulle även SVD kunna användas som en del av förarbetet på datan. Den skulle kunna ha syftet att reducera brus, tack vare sin approximering av bilden med hjälp av ett begränsat antal singulära värden. Det skulle kunna gynna Hausdorff-avstånd och template matchning eftersom de metoderna är generellt känsliga mot brus och extrempunkter. Det skulle dock kunna försvaga spisens ikon då den ofta är väldigt liten jämfört med hela ritningen och viktiga detaljer skulle kunna försvinna beroende på rank. Därför skulle framtida arbeten kunna undersöka ifall SVD kan förbättra resultat, samt undersöka vilken rank av SVD som fungerar bäst.

5.2.2 Tidsoptimerad kod

Eftersom den slutgiltiga koden tog ungefär 6.5 timmar att köra, kan man fråga sig om det är mer tidseffektivt att en människa markerar bilderna. I vårt fall där målet var att identifiera spisar, hade det räckt att markera 200 bilder för hand och tränat ett neuralt nätverk på de markerade bilderna. Å andra sidan är programmet passivt, vilket innebär att ingen mänsklig interaktion är nödvändig. En annan fördel är att samma metod kan användas för att känna igen fler objekt. En optimeringsåtgärd hade kunnat vara att skriva programmet i ett kompilerat programmeringsspråk, då det sparar mycket tid genom att kompilera innan exekvering.

5.3 Slutsats

Syftet med arbetet var att undersöka matematiska objektlokaliseringsmetoder som skulle kunna användas på planritningar för att markera spisar. Resultaten vi fick visar att det fungerar till viss del, men träffsäkerheten är såpass låg att det inte går att lita på utan att kontrollera eller markera manuellt. För normerad template matchning hade vi kunnat sätta ett tröskelvärde och fått ut en högre träffsäkerhet till priset av större andel bortfiltrerade sant-positiva resultat. De största begränsningarna med metoderna var skillnader i skala, brus, rotation och variation på spisarnas symboler.

Av de metoder som testades, var normerad template matchning med negativ padding bäst. Den hade högst träffsäkerhet och var även en av de snabbaste att exekvera. Eftersom träffsäkerheten fortfarande var runt en tredjedel, är det inte tillräckligt bra för att kunna förlita sig på, varken för att göra analyser av planritningarna eller träna ett neuralt nätverk på. Däremot kan vi med ett bra tröskelvärde få tillräckligt bra träffsäkerhet med normerad template matchning.

De Hausdorff-avstånds baserade metoderna visade sig vara väldigt känsliga mot skala och brus samt extremt krävande beräkningsmässigt. Det normerade modifierade Hausdorff-avståndet gav bäst resultat av dem, men ändå sämre och långsammare än normerad template matchning. Med en träffsäkerhet på 29.1% ser vi att metoden kan vara relevant, men inte tillräckligt effektiv för att använda i praktiken vid varken räkning av antal lägenheter eller bygga en databas.

Testerna som utfördes på CNN visade att det finns stor potential. Även vid de förhållandevis få ritningar som användes, kunde man se att den lyckades bra med att identifiera spisarna. Att använda denna metoden kräver däremot en tidskrävande manuell process av att markera ritningarna, vilket går emot vårt ursprungliga syfte att automatisera processen. Överlag kan vi se att klassiska matematiska metoder kan vara relevant vid objektigenkänning som ett första steg, särskilt normerad template matchning med ett bra tröskelvärde. Detta tillsammans med neurala nätverk kan därför ge bra resultat.

6 Samhälleliga och etiska aspekter

Arbetet ämnar att i slutändan automatisera ett tidskrävande jobb vilket är ett tudelat problem, då effektivisering och standardisering av arbetet i fråga kan komma i konflikt med att mänsklig kontroll och arbetstillfällen försvinner. Ur ett samhälleligt perspektiv kan detta ses som riskabelt då effektivisering på vissa områden kan leda till allvarliga konsekvenser på andra. Denna risk bör däremot vägas mot möjligheten att arbetstillfällen snarare förändras än försvinner. Vilket i det stora hela leder till ett större värdeskapande genom hela processen.

Användningen av objektigenkänningsmetoder på byggnadsritningar medför även en risk för skada om resultaten av dessa används okritiskt. Felklassificeringar eller utebliven igenkänning riskerar att i förlängningen leda till felaktiga beslut genom hela byggnadsprocessen. Eftersom detta arbete ej avser att tillhandahålla en färdig teknisk lösning ses detta som en risk att beakta i senare studier vid aktualisering i samband med praktisk tillämpning.

Ur ett vetenskapligt perspektiv bidrar arbetet med utvärdering, utveckling och förståelse för objektigenkänningsmetoder inom ett mindre utforskat tillämpningsområde. Specifikt ämnar inte arbetet att ha någon samhällelig nytta utan som tidigare nämnt att bidra till en ökad förståelse för vilka metoder är lämpliga för det användningsområdet.

Ritningarna som används inom projektet är av sådan karaktär att de kan betraktas som allmän handling därför de kan användas i syfte av träning och applicering på algoritmer. De innehåller heller inte personuppgifter eller känslig information, vilket innebär att arbetet inte medför några risker ut ett integritetssperspektiv.

Slutligen kommer inte arbetet fram till sådana resultat att de är tillämpbara i ett verkligt sammanhang, varför risken hos de ovan nämnda faktorerna minskar ytterligare.

Referenser

- [1] Larisa Beilina, Evgenii Karcheskii, and Mikhail Karcheskii. *Numerical Linear Algebra: Theory and Applications*. Springer, Cham, Switzerland, 2017. ISBN 978-3-319-57304-5. doi: 10.1007/978-3-319-57304-5.
- [2] Laura Boone. Industry 4.0 (fourth industrial revolution). *Salem Press Encyclopedia*, 2026.
- [3] John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8(6):679–698, 1986. doi: 10.1109/TPAMI.1986.4767851.
- [4] M.-P. Dubuisson and A.K. Jain. A modified hausdorff distance for object matching. *Proceedings of 12th International Conference on Pattern Recognition, Pattern Recognition*, 1994. doi: <https://doi.org/10.1109/ICPR.1994.576361>.
- [5] Diarmaid Geever, Tim Brophy, Imad Ali Shah, Enda Ward, Brian Deegan, Martin Glavin, and Edward Jones. Evaluation of interpolation methods for image downsampling in automotive computer vision. In *2025 IEEE International Conference on Vehicular Electronics and Safety (ICVES)*, pages 380–386, 2025. doi: 10.1109/ICVES65691.2025.11376542.
- [6] Daniel P. Huttenlocher, Gregory A. Klanderman, and William J. Rucklidge. Comparing images using the hausdorff distance. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 15(9):850–863, 1993. doi: 10.1109/34.232073.
- [7] Ramo Klevis. Object detection with the sliding window solution. In *Hands-On Java Deep Learning for Computer Vision*, pages 156–157. Packt Publishing, 2019. ISBN 978-1-5231-3251-5.
- [8] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS 25)*, pages 1097–1105, 2012.
- [9] J. P. Lewis. Fast normalized cross-correlation. In *Vision Interface*, pages 120–123, 1995.
- [10] Onur; Alsubai Shtwai Naeem, Awad Bin; Osman. Lightweight cnn for accurate brain tumor detection from mri with limited training data. *Frontiers in Medicine*, 2025. doi: <https://doi.org/10.3389/fmed.2025.1636059>.
- [11] NVIDIA Corporation. Cuda toolkit, 2024. URL <https://developer.nvidia.com/cuda/toolkit>. NVIDIA Developer Portal.
- [12] OpenCV. Geometric image transformations. https://docs.opencv.org/3.4/da/d54/group__imgproc__transform.html, . Accessed: 2026-04-23.
- [13] OpenCV. Template matching (cv::templatematchmodes). https://docs.opencv.org/4.x/df/dfb/group__imgproc__object.html, . Accessed: 2026-04-11.
- [14] Keiron O’Shea and Ryan Nash. An introduction to convolutional neural networks. *CoRR*, abs/1511.08458, 2015. URL <http://arxiv.org/abs/1511.08458>.
- [15] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016. doi: 10.1109/CVPR.2016.91.
- [16] Richard Szeliski. *Computer Vision*. Texts in Computer Science. Springer Cham, 2022. doi: 10.1007/978-3-030-34372-9. pp. 33–34.
- [17] Guixian Wu and Yuancheng Li. Non-maximum suppression for object detection based on the chaotic whale optimization algorithm. *Journal of Visual Communication and Image Representation*, 74:102985, 2021. ISSN 1047-3203. doi: 10.1016/j.jvcir.2020.102985. URL <https://www.sciencedirect.com/science/article/pii/S1047320320302042>.

AI användande

Under arbetets gång har generativa AI-verktyg används som stöd till rapportskrivningen. Verktugen har främst använts för språklig bearbetning av text, strukturering av text, felsökning av kod samt formulering av matematiska resonemang. Allt innehåll som genererats av AI-verktyg har granskats, bearbetats och kontrollerats av författarna. Verktugen har inte använts för att generera resultat eller ersätta författarnas bedömningar. Ett undantag är översättningen av sammandraget, där den engelska motsvarigheten är genererad utifrån det svenska sammandraget. Författarna ansvarar därför för rapportens innehåll, slutsatser och källhänvisningar.

A Appendix 1 – teori

A.1 Modifierat Hausdorff-avstånd

Algorithm 1 Modifierat Hausdorff-avstånd givet mängderna A, B

```
1: function UNDIRECTEDMODIFIEDHAUSDORFF( $A, B$ )
2:    $sumAB \leftarrow 0$ 
3:   for all  $a \in A$  do
4:      $minDist \leftarrow \infty$ 
5:     for all  $b \in B$  do
6:        $minDist \leftarrow \min(minDist, \|a - b\|)$ 
7:     end for
8:      $sumAB \leftarrow sumAB + minDist$ 
9:   end for
10:   $h_{AB} \leftarrow sumAB/|A|$ 
11:   $sumBA \leftarrow 0$ 
12:  for all  $b \in B$  do
13:     $minDist \leftarrow \infty$ 
14:    for all  $a \in A$  do
15:       $minDist \leftarrow \min(minDist, \|b - a\|)$ 
16:    end for
17:     $sumBA \leftarrow sumBA + minDist$ 
18:  end for
19:   $h_{BA} \leftarrow sumBA/|B|$ 
20:  return  $\max(h_{AB}, h_{BA})$ 
21: end function
```

Sats 1. Modifierat Hausdorff-avstånd har tidskomplexiteten $O(n^2)$

Bevis. Antag $|A| = N$ och $|B| = M$. Algoritmen innehåller två nästlade loopar: Den yttre körs N gånger, och den inre M . Därav görs $N \cdot M$. Varje beräkning görs i konstant tid $O(1)$, vilket ger en tidskomplexitet

$$O(NM) + O(NM) = O(NM)$$

Om vi vidare antar att $N \approx M$, får vi $NM \approx n^2$ vilket ger en total tidskomplexitet på

$$O(NM) = O(n^2)$$

Algoritmens komplexitet är alltså kvadratisk givet att mängderna har ungefär samma storlek. $\square$

A.2 Template matching algoritm

Sats 2. Ekvation 10 är lika med cosinus av vinkeln mellan de två matriserna i vektoriserat format.

Bevis. Antag att vi har matriserna $T \in \mathbb{R}^{m \times n}$ och $I \in \mathbb{R}^{m \times n}$. Låt $\mathbf{t}, \mathbf{i} \in \mathbb{R}^{mn}$ beteckna de vektoriserade representationerna av T respektive I , erhållna genom att stapla matriselementen. Då kan summan i täljaren skrivas som skalärprodukten mellan de två vektorerna;

$$\sum_{x', y'} (T(x', y') \cdot I(x + x', y + y')) = \mathbf{t} \cdot \mathbf{i}$$

Vidare kan nämnaren skrivas i form av den Euklidiska normen av de två vektorerna:

$$\sqrt{\sum T^2 \cdot \sum I^2} \Leftrightarrow \|\mathbf{t}\| \cdot \|\mathbf{i}\|$$

Insättning i Ekvation 10 ger då;

$$R(x, y) = \frac{\mathbf{t} \cdot \mathbf{i}}{\|\mathbf{t}\| \cdot \|\mathbf{i}\|}$$

Enligt definition kan skalärprodukten mellan två vektorer skrivas som

$$\mathbf{t} \cdot \mathbf{i} = \|\mathbf{t}\| \cdot \|\mathbf{i}\| \cos \theta$$

vilket insatt i tidigare samband ger att

$$R(x, y) = \frac{\mathbf{t} \cdot \mathbf{i}}{\|\mathbf{t}\| \cdot \|\mathbf{i}\|} = \cos \theta$$

□

Sats 3. *Normerad template matchning har linjär tidskomplexitet.*

Bevis. Given en bild $I \in \mathbb{R}^{m \times n}$ och ett referensobjekt (Template) $T \in \mathbb{R}^{w \times h}$.

Referensobjekt placeras på alla giltiga positioner i bilden

$$(m - w + 1)(n - h + 1)$$

vilket betecknar alla unika positioner inuti I i vilka T kan placeras.

Vid varje position varje position jämförs varje pixel i referensbilden vilket är $w \cdot h$ operationer.

Alltså blir den totala mängden operationer:

$$O(m, n) = (m - w + 1)(n - h + 1) \cdot (wh) = (mn - mh + m - wn + wh - w + n - h + 1)(wh)$$

och eftersom den största ordningens termer dominerar får vi slutligen

$$= (mn) \cdot (wh).$$

Givet att w, h är konstanta och $w \times h \ll m \times n$ betecknas $w \times h$ som en konstant C , vilket insatt i tidigare samband ger:

$$O(m, n) = (mn \cdot C) = O(mn).$$

□

B Appendix 2 – källkod

B.1 Hausdorff

```
def image_to_points(img, threshold=175):
    """
    Tar en OpenCV-bild (numpy array) och returnerar en Nx2-array
    med (y, x)-punkter för pixlar som är 'på'.
    """

    if img is None:
        return np.empty((0, 2))

    # Konvertera till gråskala om den är färg
    if len(img.shape) == 3:
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    else:
        gray = img

    # Binarisera
    binary = gray < threshold

    # Hämta koordinater
    points = np.column_stack(np.nonzero(binary))

    return points


def modified_hausdorff_distance(A, B):
    """
    A, B: Nx2 respektive Mx2 numpy-arrayer
    """

    tree_A = cKDTree(A)
    tree_B = cKDTree(B)

    # Avstånd från A till närmaste punkt i B
    dist_A_to_B, _ = tree_B.query(A)
    mean_A_to_B = np.mean(dist_A_to_B)

    # Avstånd från B till närmaste punkt i A
    dist_B_to_A, _ = tree_A.query(B)
    mean_B_to_A = np.mean(dist_B_to_A)

    return max(mean_A_to_B, mean_B_to_A)
```

B.2 Kantdetektion och gaussisk utjämning

```
def edge_detection(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    blur = cv2.GaussianBlur(gray, (5, 5), 0)
    edges = cv2.Canny(blur, threshold1=150, threshold2=250)

    # Canny ger: kanter = vit (255), bakgrund = svart (0)
    # Vi vill ha tvärtom → invertera
    edge_map = cv2.bitwise_not(edges)

    return edge_map
```

B.3 Sliding window

```
import cv2
import numpy as np
import os
import matplotlib.pyplot as plt
from time import time, strftime, gmtime
from edge_detect import modified_hausdorff_distance, image_to_points, edge_detection
import sys
import csv
from concurrent.futures import ProcessPoolExecutor, as_completed

# -----
# Helpers
# -----
def sliding_window(image, window_size, stride, offset_x=0, offset_y=0):
    h, w = image.shape[:2]
    win_h, win_w = window_size

    for y in range(offset_y, h - win_h + 1, stride):
        for x in range(offset_x, w - win_w + 1, stride):
            patch = image[y:y + win_h, x:x + win_w]
            yield (x, y, patch)

def plot_points(points, size=20):
    y, x = zip(*points)
    plt.scatter(x, y, marker="s", s=size)

def build_ref_points(reference_path, window_size):
    ref_images_points = []
    valid_ext = (".png", ".jpg", ".jpeg", ".bmp", ".tif", ".tiff")

    for ref_img_name in os.listdir(reference_path):
        if not ref_img_name.lower().endswith(valid_ext):
            continue

        ref_image = cv2.imread(os.path.join(reference_path, ref_img_name))
        if ref_image is None:
            continue

        ref_image_small = cv2.resize(ref_image, window_size,
                                     interpolation=cv2.INTER_AREA)

        # Om du vill använda kanter, slå på detta:
        ref_image_small = cv2.Canny(ref_image_small, 100, 200)

        ref_image_points = image_to_points(ref_image_small)

        # (valfritt) om en referens råkar ge tomt punktmoln, skip:
        if len(ref_image_points) == 0:
            continue

        ref_images_points.append(ref_image_points)
```

```

return ref_images_points

def run_scan(image, ref_images_points, window_size, stride,
             interesting_threshold, interest_radius,
             print_status_fn=None, get_time_fn=None):
    result_image = image.copy()

    distances = []
    windows_of_interest = []
    i = 0
    prev_time = time()

    h, w = image.shape[:2]
    total_windows_number = max(
        1,
        ((w - window_size[0]) // stride + 1) * ((h - window_size[1]) // stride + 1)
    )

    for x_offset, y_offset, patch in sliding_window(image, window_size,
                                                    stride, offset_y=0, offset_x=0):
        patch_edges = patch # eller: edge_detection(patch)
        patch_points = image_to_points(patch_edges)
        i += 1

        # Progress bar
        if print_status_fn is not None and get_time_fn is not None:
            if i % 2000 == 0 and time() - prev_time > 1:
                prev_time = time()
                percentage = round((i / total_windows_number) * 100, 1)
                print_status_fn(f"{get_time_fn()} | win={window_size[0]} | {percentage}%")

        # Snabba skip om patch är helt vit eller helt svart
        if len(patch_points) == 0:
            continue
        if np.all(patch_edges == 255):
            continue

        # Min distance mot alla referenser
        min_dist = np.inf
        for ref_img_pts in ref_images_points:
            distance = modified_hausdorff_distance(patch_points, ref_img_pts)

            ##Normalizing the diagonal of the window size
            #diag = np.sqrt(window_size[0]**2 + window_size[1]**2)
            #distance = distance / diag

            if distance < min_dist:
                min_dist = distance

        distances.append(min_dist)

        # Samla detections
        if min_dist < interesting_threshold:

```

```

        for j, t in enumerate(windows_of_interest):
            ix, iy, idist, number_of_windows = t
            if (ix - interest_radius < x_offset < ix + interest_radius and
                iy - interest_radius < y_offset < iy + interest_radius):
                if min_dist < idist:
                    windows_of_interest[j] = [x_offset, y_offset,
                                                min_dist, number_of_windows]
                    windows_of_interest[j][3] += 1
                break
            else:
                windows_of_interest.append([x_offset, y_offset, min_dist, 1])

# Rita detections
# Välj top 5 (lägsta distans)
top_k = 1
top_windows = sorted(windows_of_interest, key=lambda t: t[2])[:top_k]

# Rita endast top 5
for t in top_windows:
    ix, iy, idist, number_of_windows = t

    cv2.putText(
        result_image, f"{number_of_windows}",
        (ix + 2, iy + (window_size[1] // 3)),
        cv2.FONT_HERSHEY_SIMPLEX, 0.4, (0, 255, 0)
    )
    cv2.putText(
        result_image, f"{round(idist, 2)}",
        (ix + 2, iy + (window_size[1] // 3) * 2),
        cv2.FONT_HERSHEY_SIMPLEX, 0.4, (255, 0, 0)
    )
    cv2.rectangle(
        result_image,
        (ix, iy),
        (ix + window_size[0] - 2, iy + window_size[1] - 2),
        (0, 0, 255), 2
    )

sorted_dists = sorted(distances) if len(distances) else []
best5 = sorted_dists[:5]

best_det = None
if windows_of_interest:
    best_det = min(windows_of_interest, key=lambda t: t[2])

return result_image, windows_of_interest, best5, best_det

def run_one_job(image_path, window_size, reference_path, out_dir,
                stride_base, interesting_threshold, interest_radius):
    """
    Kör en (bild, window_size)-kombination.
    Returnerar en CSV-rad: [image, window_size, best_dist, best_x, best_y,
    best_count, num_detections, best_overall_for_image]
    """
    image = cv2.imread(image_path)
    if image is None:
        return None

```

```

base_name = os.path.splitext(os.path.basename(image_path))[0]

# Bygg referenser (i varje process). Enklast och funkar bra.
ref_images_points = build_ref_points(reference_path, window_size)
if len(ref_images_points) == 0:
    return [base_name, window_size[0], "", "", "", "", 0, "FALSE"]

stride_for_size = max(stride_base, window_size[0] // 8)

result_image, windows_of_interest, best5, best_det = run_scan(
    image=image,
    ref_images_points=ref_images_points,
    window_size=window_size,
    stride=stride_for_size,
    interesting_threshold=interesting_threshold,
    interest_radius=interest_radius,
    print_status_fn=None,
    get_time_fn=None
)

out_path = os.path.join(out_dir, f"{base_name}_win{window_size[0]}.png")

os.makedirs(out_dir, exist_ok=True) # viktigt när processer spawnas
out_path = os.path.join(out_dir, f"{base_name}_win{window_size[0]}.png")

ok = cv2.imwrite(out_path, result_image)
if not ok:
    return [base_name, window_size[0], "IMWRITE_FAIL", out_path,
            "", "", 0, "FALSE"]

num_det = len(windows_of_interest)
if best_det is None:
    return [base_name, window_size[0], "", "", "", "", num_det, "FALSE"]

bx, by, bdist, bcount = best_det
return [base_name, window_size[0], round(float(bdist), 6), int(bx),
        int(by), int(bcount), num_det, "FALSE"]

# -----
# Main config
# -----
if __name__ == "__main__":
    stride_base = 3
    interesting_threshold = 1.5
    interest_radius = 10

    ritning_dir = "Ritningar"
    reference_path = "reference_images"
    valid_ext = (".png", ".jpg", ".jpeg", ".bmp", ".tif", ".tiff")

    win_min, win_max, win_step = 34, 70, 2
    window_sizes = [(s, s) for s in range(win_min, win_max + 1, win_step)]

    out_dir = "results"
    out_dir = os.path.abspath(out_dir)
    os.makedirs(out_dir, exist_ok=True)

```

```

print("Saving results to:", out_dir)
os.makedirs(out_dir, exist_ok=True)

# Sanity checks
if not os.path.isdir(ritning_dir):
    raise FileNotFoundError(f"Hittar inte mappen: {ritning_dir}")

if not os.path.isdir(reference_path):
    raise FileNotFoundError(f"Hittar inte mappen: {reference_path}")

image_paths = [
    os.path.join(ritning_dir, f)
    for f in os.listdir(ritning_dir)
    if f.lower().endswith(valid_ext)
]
if len(image_paths) == 0:
    raise FileNotFoundError(f"Inga bildfiler hittades i {ritning_dir}
        (kollar {valid_ext})")

ref_files = [
    f for f in os.listdir(reference_path)
    if f.lower().endswith(valid_ext)
]
if len(ref_files) == 0:
    raise FileNotFoundError(f"Inga referensbilder hittades i
        {reference_path} (kollar {valid_ext})")

starttime = time()

def print_status(msg: str):
    sys.stdout.write("\r" + msg + "\033[K")
    sys.stdout.flush()

def get_time():
    return strftime('%H:%M:%S', gmtime(time() - starttime))

# -----
# CSV setup + caching refs
# -----
summary_csv_path = os.path.join(out_dir, "summary.csv")
csv_header = [
    "image",
    "window_size",
    "best_dist",
    "best_x",
    "best_y",
    "best_count",
    "num_detections",
    "best_overall_for_image"
]

```

```

# -----
# Run
# -----
# -----
# Run (multiprocessing)
# -----
max_workers = os.cpu_count() - 1 if os.cpu_count() and os.cpu_count() > 1 else 1

jobs = [(img_path, ws) for img_path in image_paths for ws in window_sizes]

rows_by_image = {}

with ProcessPoolExecutor(max_workers=max_workers) as ex:
    futures = [
        ex.submit(
            run_one_job,
            img_path, ws, reference_path, out_dir,
            stride_base, interesting_threshold, interest_radius
        )
        for (img_path, ws) in jobs
    ]

    done = 0
    total = len(futures)

    for fut in as_completed(futures):
        row = fut.result()
        done += 1

        if row is None:
            print_status(f"{get_time()} | klara {done}/{total} | row=None")
            continue

# En rad per färdigt jobb:
    print_status(f"{get_time()} | klara {done}/{total} | {row[0]}
                win={row[1]} best={row[2]}")

    img = row[0]
    rows_by_image.setdefault(img, []).append(row)

    print() # ny rad efter status

# Markera bästa window_size per bild (TRUE)
for img, rows in rows_by_image.items():
    best = None
    for r in rows:
        if r[2] == "":
            continue
        if best is None or float(r[2]) < float(best[2]):
            best = r
    if best is not None:
        for r in rows:
            if r[1] == best[1]:
                r[-1] = "TRUE"

# Skriv CSV i huvudprocessen

```



```

with open(summary_csv_path, "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f, delimiter=";")
    writer.writerow(csv_header)

    for img in sorted(rows_by_image.keys()):
        # sortera rader efter window_size
        for r in sorted(rows_by_image[img], key=lambda x: x[1]):
            writer.writerow(r)

print_status(f"{get_time()} | KLAR")
print(f"\nSkrev CSV: {summary_csv_path}")

```

B.4 Template matching

```

import cv2
import numpy as np
import time
import os
from matplotlib import pyplot as plt

ritningar = os.listdir("Ritningar")
for i, img1_filename in enumerate(ritningar):
    # ===== 1. Läs bilder =====
    img1 = cv2.imread(f"Ritningar/{img1_filename}")

    gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)

    # ===== 2. Kantdetektion =====
    edges1 = cv2.Canny(gray1, 30, 100)

    # ===== 3. Template matching för varje rotation =====
    all_matches = []

    start = time.time()

    for refimg_name in os.listdir("tempmatch_refbilder"):
        img2 = cv2.imread(f"tempmatch_refbilder/{refimg_name}")
        gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
        edges2 = cv2.Canny(gray2, 30, 100)

        # ===== 2b. Maskera 10% av kanterna på referensbilden =====
        h2, w2 = edges2.shape[:2]
        margin_y = int(h2 * 0.10)
        margin_x = int(w2 * 0.10)
        mask2 = np.zeros_like(edges2)
        mask2[margin_y:h2-margin_y, margin_x:w2-margin_x] = 255
        edges2_masked = cv2.bitwise_and(edges2, mask2)

        # ===== 2c. Skapa roterade versioner =====
        rotations = {
            0: edges2_masked,
            90: cv2.rotate(edges2_masked, cv2.ROTATE_90_CLOCKWISE),
            180: cv2.rotate(edges2_masked, cv2.ROTATE_180),
            270: cv2.rotate(edges2_masked, cv2.ROTATE_90_COUNTERCLOCKWISE),
        }

    for angle, tmpl_edges in rotations.items():

```

```

h, w = tpl_edges.shape[:2]
short_side = min(w, h)

for target_px in range(35, 70):
    scale = target_px / short_side
    tw = int(w * scale)
    th = int(h * scale)

    if tw < 5 or th < 5:
        continue
    if tw > edges1.shape[1] or th > edges1.shape[0]:
        continue

    resized_tmpl = cv2.resize(tmpl_edges, (tw, th),
                              interpolation=cv2.INTER_AREA)

    margin_py = int(th * 0.10)
    margin_px = int(tw * 0.10)
    patch_mask = np.zeros((th, tw), dtype=np.uint8)
    patch_mask[margin_py:th-margin_py, margin_px:tw-margin_px] = 255
    resized_tmpl = cv2.bitwise_and(resized_tmpl, patch_mask)

    result = cv2.matchTemplate(edges1, resized_tmpl,
                               cv2.TM_CCOEFF_NORMED)

    elapsed = time.time() - start
    _, max_val, _, max_loc = cv2.minMaxLoc(result)

    result_copy = result.copy()
    for _ in range(10):
        _, max_val, _, max_loc = cv2.minMaxLoc(result_copy)
        # Spara även originalskalan (tw, th) och scale för att
        # rita utan erosion
        all_matches.append((max_val, max_loc, (tw, th), angle,
                           scale, refimg_name))
        x, y = max_loc
        result_copy[max(0, y - th):y + th,
                    max(0, x - tw):x + tw] = -1

total = time.time() - start
#print(f"\nTotal tid: {total:.2f}s")

# ===== 4. Hitta bästa match =====
all_matches.sort(key=lambda m: m[0], reverse=True)
best_score, best_loc, best_dim, best_angle, best_scale,
best_ref = all_matches[0]

print(f"[{i+1}/{len(ritningar)}] Bästa match: score={best_score:.3f},
      rotation={best_angle}°, position={best_loc}, ref={best_ref}")

# ===== 5. Beräkna originalstorlek utan erosion =====
# Roterar img2 på samma sätt som bästa matchen
rotate_flags = {
    0: None,
    90: cv2.ROTATE_90_CLOCKWISE,
    180: cv2.ROTATE_180,
    270: cv2.ROTATE_90_COUNTERCLOCKWISE,

```

```

}
if rotate_flags[best_angle] is not None:
    img2_rotated = cv2.rotate(img2, rotate_flags[best_angle])
else:
    img2_rotated = img2.copy()

# Skala referensbilden till samma storlek som bästa matchen (ingen erosion)
orig_h, orig_w = img2_rotated.shape[:2]
tw_orig = int(orig_w * best_scale)
th_orig = int(orig_h * best_scale)
img2_scaled = cv2.resize(img2_rotated, (tw_orig, th_orig),
    interpolation=cv2.INTER_AREA)

# ===== 6. Visualisera =====
vis = img1.copy()
tl = best_loc
br = (tl[0] + tw_orig, tl[1] + th_orig)

cv2.rectangle(vis, tl, br, (0, 0, 255), 2)
cv2.putText(vis, f"best_score:.2f", (tl[0], max(tl[1] - 8, 15)),
    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)

# Spara
cv2.imwrite(f"tempmatched_ritningar/tm_{img1_filename}", vis)

```

C Appendix 3 – exempelritning

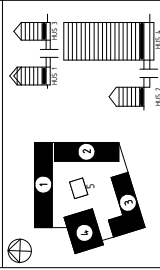
HÄNVISNINGAR

För förklaringar och föreskrifter, se ritning A-01-0-000.

SEKTÖR:	A	A-40-2-0010
	B	A-40-2-0020
	C	A-40-2-0030
	D	A-40-2-0040 och A-40-2-0050

B	PM F	20323	PMOH
A	PM C	20426	PMOH
DET	ANT	INTERGEN AVSES	SIGN

BYGGLOVHANDLING



KALLEBÄCK KV. 6

KALLEBÄCK KV. 9
Mejerigatan, Kallebäck 18:14, Göteborg



Wallenstam AB, 401 84 Göteborg
Besöksadress Kungälvsvägen 2
Tel: 031-20 00 00

WALLENSTAM

	PORTAL	TELEFON
A) CREAM Architects / Voten Konsult AB	T 0737-467686	
K) Sjöberg AB	T 031-231390	
E) Elyscy Göteborg AB	T 0741-709020	
V) AFRY AB	T 030-5050000	
V) Sjöstrand Göteborg AB	T 031-7065300	
S) Sparrskonsulterna i Göteborg AB	T 031-907870	
B) Bengt Dahlgren Brand & Risk AB	T 031-2925000	
L) Liljevalli	T 090-2059880	
C) AEC AB	T 090-892000	
Engineering AB		
2023	STADEN EK/ÅH	HANDELSEN F. KÄRIN

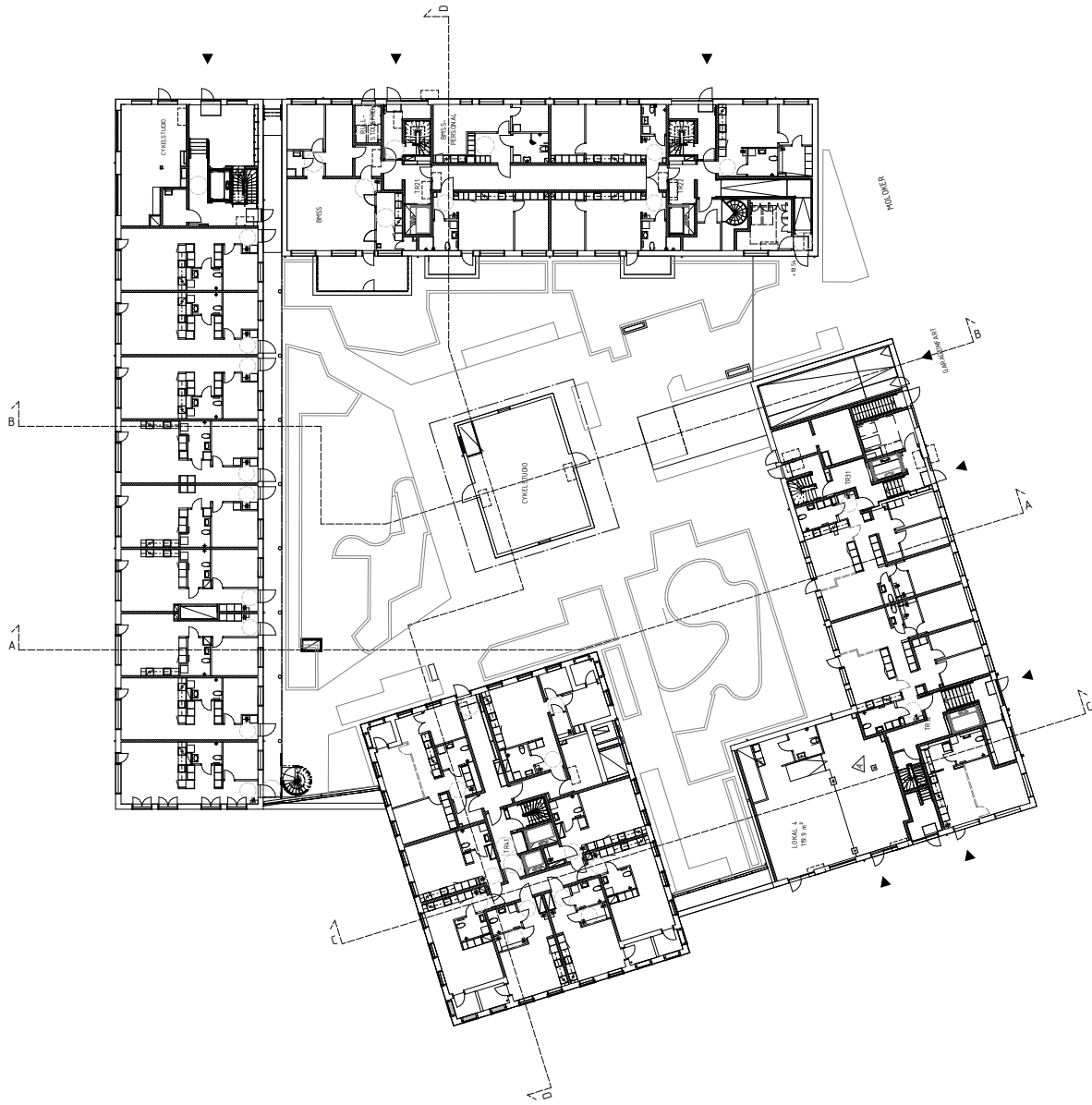
DA TUM	LIP PORAGASSANSV ABIG
2021-10-11	FIIP KARIÉN

NYBYGGNAD FLERBOSTADSHUS

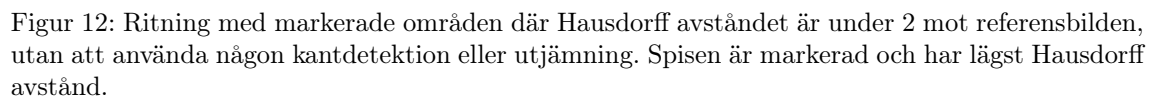
PLAN 11 ÖVERSIKT

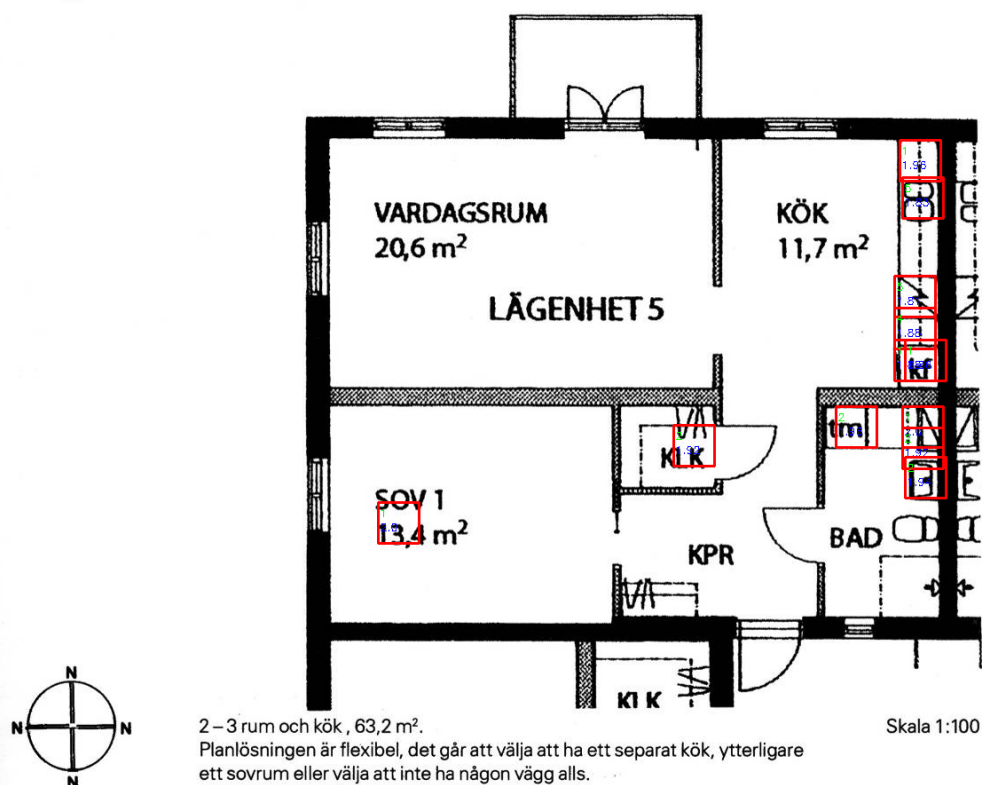
1000

SCALA
1:200
NUMERO
A-40-1-110



D.1 Kantdetektion



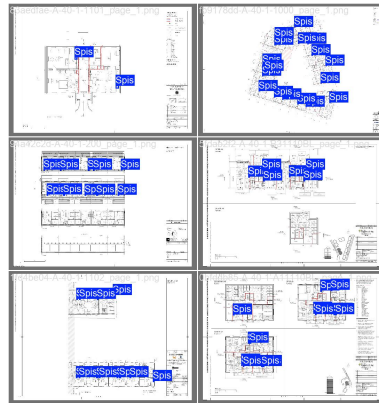


Figur 13: Ritning med markerade områden där Hausdorff avståndet är under 2 mot referensbilden. Varje fönster förbehandlades med gaussisk utjämning och Canny kantdetektionsalgoritm före Hausdorff avståndet beräknades.

D.2 CNN



Figur 14: Identifiering av en spis, förenklad bild



Figur 15: Resultat på fullskalig ritning